

CS132 Coursework

Archie Harrodine
2127935

Contents

1	Question 1.	2
1.1	a.	2
1.1.1	Decomposition	2
1.1.2	Input validation	2
1.1.3	Powerset algorithm	3
1.1.4	User input	5
1.1.5	Powerset implementations	6
1.2	b.	7
1.2.1	Execution	7
1.2.2	Testing	10
1.2.3	Conclusion	13
2	Question 2.	13
2.1	a.	13
2.1.1	Decomposition	14
2.1.1.1	Preliminary knowledge	14
2.1.1.2	Function details	14
2.1.2	Design	16
2.1.2.1	Create File	16
2.1.2.2	Delete File	17
2.1.2.3	Show File	17
2.1.2.4	Copy File	17
2.1.2.5	Append Line	18
2.1.2.6	Show Line	19
2.1.2.7	Delete Line	20
2.1.2.8	Insert Line	22
2.1.2.9	Number of lines	24
2.1.3	User input	24
2.1.3.1	Method of input	24
2.1.3.2	Scanf vs fgets	25
2.1.3.3	Interpreting the input	25
2.1.3.4	Commands and functions	27
2.1.4	Changelog design	30
2.1.5	Testing	31
2.1.5.1	Erroneous inputs	31
2.1.5.2	Size and memory allocation	33
2.1.5.3	Function edge cases	34
2.1.5.4	Execution	34
2.1.6	Conclusion	36
2.2	b.	37
2.2.1	Decomposition	37
2.2.2	List files	38
2.2.3	Evaluate line	39

2.2.3.1	Decomposition	39
2.2.3.2	Worked examples	41
2.2.3.3	Design	42
2.2.4	Testing	45
2.2.5	Conclusion	47

3	Bibliography	47
---	--------------	----

1 Question 1.

The powerset of a set S is the the set of all subsets of S .

1.1 a.

Write a program that takes a set S as input and outputs the powerset of S . You should ensure that your program is well documented and any error cases are handled appropriately. Clearly state any assumptions.

1.1.1 Decomposition

This problem can be deconstructed into several facets, relying on the definition of a power set. Firstly, in the context of a program, a regular set can be comparable to an array; in which it is a series of elements. The powerset of a set is then defined as all sets that are contained within said set. The best example to demonstrate this, as well as act as our base case is the set

$$\{a, b, c\}$$

This has the powerset

$$\{\{\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$$

This is because each element of the powerset, is a subset of the original set.

For the sake of notation, when referring to the original set, it will be referred to as S , and the powerset of S will be 2^S . This is because, for a set of size n , its powerset is 2^n .

When looking at the problem, there are a few areas to put focus upon:

1. Entering the initially entered set.
2. Validating the initially entered set.
3. Systematically going through the set to output the powerset.

These initially seem simple conditions to suffice, there are a few complications and design choices that need to be made. The simplest task is to have the user enter the set. Each element will be considered as a string, as to not exclude any inputs. Here, each element entered will be added to an array which will act as a set. However, sets are not arrays, and this creates complications in the validation step.

1.1.2 Input validation

Sets differ from arrays as sets are unordered, and there can be no repeating elements. That is to say that

$$\{b, c, a\} \equiv \{a, b, c\}$$

and,

$$\{a, a, b, c\} \equiv \{a, b, c\}$$

This creates a problem as there is nothing stopping the user from entering the same element into a set multiple times. There are two solutions to this: validating during input, or post input. This would be done by performing a linear search throughout the set, and identifying if any elements are the same. On the one hand, this search would be performed after each input, and informing the user that the input has already been entered. Alternatively, it could do the search on every element after it has been entered. The first option has the advantage that the user

would be informed that that element was rejected in real time, and hence making the interface and interaction of the machine and user to be more effective. However, the second option not only is simpler, but requires less comparisons of elements on average. This can be inferred by the fact that if we have the full set, then all copies of a single element will be removed in the first pass, removing future comparisons to make; whereas the other approach will remove repeated elements, but then if that same repeated element is added in the next cycle, it will need to make the comparisons once more.

The decider for which of the two methods, as in real life, came down to the resources given to me by C. Whilst Java and Python can intuitively have variable number elements of a list; in C they are limited to a static array, meaning that once an item is appended, it can't be removed. Whilst elements can be changed, the system to do this under the pretext of removing an element, requires the indexes of the elements in the list to be decremented, and frankly, it is more effort than it is worth. The simpler solution, is to integrate a test for each entry, comparing it to every item in the array. If it doesn't appear, then it can be added.

1.1.3 Powerset algorithm

Following the input comes the big part of the program, and that is the method of outputting the powerset. Whilst initially I was drawn to using a two dimensional array, in which another array would be made to reflect the input. This is not the optimal method though as C's arrays are static, requiring set data types, and sizes of 2D arrays. Hence, I scrapped that method, and began to search online for a new approach.

The first website explored was GeeksforGeeks, in which they proposed this algorithm (algorithm below is written in pseudocode, which is my understanding of the method, without a specific language).

```
1 powerset_size = 2^set_size
2 for index=0 to powerset_size
3     for i=0 to set_size
4         if ith bit in index is 1 then
5             output set[i]
6     output ""
7     next i
8 next counter
```

This method is something that is far harder to understand on just a first glance; I say as I had to work through it to see how it made sense.

```
1 set = [a,b,c]
2 set_size = 3
3 powerset_size = 8
4
5 counter = 0000 #0 in binary
6     i = 0001 #1 shifted by 0
7     --
8     i = 0010 #1 shifted by 1
9     --
10    i = 0100 #1 shifted by 2
11    --
12    i = 0100 #1 shifted by 3
13 output ""
14 counter = 0001 #1 in binary
15     i = 0001
16     output "a"
17     i = 0010
18     --
19     i = 0100
20     --
21 output ""
22 counter = 0010 #2 in binary
23     i = 0001
24     --
25     i = 0010
26     output "b"
27     i = 0100
28     --
29 output ""
30 counter = 0011 #3 in binary
31     i = 0001
```

```

32     output "a"
33     i = 0010
34     output "b"
35     i = 0100
36     --
37 output ""
38 ...

```

This continues, and ends with the final output of:

$$\{\{a\}, \{b\}, \{a, b\}, \{c\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$$

This isn't perfect as it doesn't include the empty set, but otherwise it works well, outputting all meaningful combinations. This method requires bitwise manipulation, with many comparisons/if statements that do not output anything. The reason this method works is because counting up in binary gives all combinations, in a way it works out that certain binary values correspond to each set:

```

1 000 : {}
2 001 : {a}
3 010 : {b}
4 011 : {a,b}
5 100 : {c}
6 101 : {a,c}
7 110 : {b,c}
8 111 : {a,b,c}

```

This method is more complicated, however it makes great use of binary representation, perfect for a lower level language like C, but this requires understanding of binary manipulation in C.

The alternative method proposed is using the recursive approach. This defines a function which is recursively used, varying the index, and the currently stored subset. The pseudocode below demonstrates the algorithm to my understanding:

```

1 function powerset(set, index, subset)
2     if index == set_size then
3         output subset
4         return
5     powerset(set, index+1, subset + set[index])
6     powerset(set, index+1, subset)
7 end function
8 powerset(set, 0, "")

```

This method works in a slightly different fashion, that similarly requires a bit of understanding to see why it outputs everything required.

```

1 set = [a,b,c]
2 set_size = 3
3 powerset(set, 0, "")
4     powerset(set, 1, "a")
5         powerset(set, 2, "ab")
6             powerset(set, 3, "abc")
7                 output "abc"
8             powerset(set, 3, "ab")
9                 output "ab"
10        powerset(set, 2, "a")
11            powerset(set, 3, "ac")
12                output "ac"
13            powerset(set, 3, "a")
14                output "a"
15    powerset(set, 1, "")
16    ...

```

This continues on recursing until it eventually gives the full output

$$\{\{a, b, c\}, \{a, b\}, \{a, c\}, \{a\}, \{b, c\}, \{b\}, \{c\}\}$$

This other method is stronger as it has far less comparisons that do not amount to anything, but it comes with the many problems that come with recursion. Each time the function recurses, creates a new recursion stack, using

up an extortionate amount of memory, especially when we consider the number of times it must recurse. By using some simple inference, we can find that the function is called $2^{(n+1)} - 1$ times, with the max depth being $n + 1$ one times (where n is the size of the set). What this means is that, this is not very efficient for memory, using it up very fast, and likely reaching the maximum recursion depth in C. Another limitation with this method is the uncertainty that is using the concatenation of strings; just one other problem to consider.

Hence, I can either implement the algorithm that uses bit manipulation, or I can try to recreate the recursion algorithm, only using for and while loops. The difficulty in recreating the the algorithm is that it recurses two separate times, meaning it is not just a single variable that needs to be changed. The problem lies in how there is no effective way to traverse through the set without using recursive methods; or at least which I can find. However, there was one other area to explore, and that is whether recursion depth would actually be an issue. See, the largest problem at hand is concerning memory, however if we have a look at the code contained in the function, we can find that there aren't any variables declared within the function, there only being an output. Therefore we could conclude that there are few limitations present. The one downside to this, is that there is such a thing as a recursion limit (maximum number of times the program can recurse). The recursion limit varies from program, and so it is hard to conclude what is the correct algorithm, without actually programming it and testing it.

1.1.4 User input

The first, and arguable the most difficult part of the program surrounds getting the user input. The way this will be implemented is through the use of the 2D array that C offers. The set will be the 2D array, with it being an array of elements of the list. To avoid any difficulty of ending the the function, the first input will be the size of the set, and the user will enter that many elements. As discussed earlier, we will check after each inputted element, whether it has been entered already.

```

1 char** getSet(int size)
2 {
3     int i;
4     int j;
5
6     char **arr;
7
8     arr = malloc(size * sizeof(char*));
9     for (int i = 0; i < size; i++){
10         arr[i] = malloc((20) * sizeof(char));
11     }
12
13     printf("Enter %d elements:\n",size);
14     for (i=0; i<size; ++i){
15         scanf("%s",&arr[i]);
16         int found = 0;
17         char* curr = arr[i];
18         for (j=0; j<i; j++){
19             char* temp = arr[j];
20             if (temp == curr){
21                 found = found + 1;
22             }
23         }
24         if (found > 0){
25             printf("There cannot be repeats of elements, enter another one:\n");
26             i = i-1;
27         }
28     }
29
30     return arr;
31 }

```

Note: the variable size is the size of the set that the user has entered. This function can be broken down into several sections. Firstly, (after all of the variables are declared), we use malloc to to define an area of memory for the set to be stored. As we are making a 2D array we need to do this twice. For every index of the set, the element has 20 characters allocated to it. This means there is a limit of the size of the element, but this may be up for change.

Once the array is defined, we start the for loop in which the user will be prompted to enter a new element continuously. As we use scanf, we limit ourselves as it means that strings are limited to the first space in the input.

This once again may be up for change. Then once we have the value entered, the i th index of the set, we then perform a check. We define a temporary variable of `curr`, in which it holds the currently checked value. We then loop through the list until it has reached i , each element being compared against `curr`. If at any point, the counter of found will be incremented. This is a linear search at its most simple.

Once this loop has occurred to completion, we finally check whether found is 0; as if it is it never had to increment, and hence never encountered any repeats. If it is greater than 0, then it returns an error message and manually decrements the counter i , prompting the user for a new input on the same index. Finally once the original loop is broken, the function returns the set it has made.

By keeping this function separate, it allows for much nicer and more concise code. Beyond that superficial aspect, it will help dramatically with testing and debugging, letting me test that the input is actually of the form that I would want.

Concerning the efficiency of this program, it falters in certain areas, as for larger and larger sets it has more and more values to input, store and compare against the whole array. To emphasise this, if we said that it was an input of size n , then we could gather that it loops n times; and within that loop it then must loop 1 time, 2 times, 3, all the way up to n . As this sums together we can conclude it has time complexity:

$$O\left(\sum_{k=1}^n k\right) = O\left(\frac{1}{2}n(n-1)\right) = O(n^2)$$

Which as that is just the process to get the inputs, for large numbers it may not be effective.

Speaking of the size of the inputs though, there is something consider, and that is the maximum size of the array. The reason this is important, as C does not have unlimited memory, and we cannot presume that it does, so we should put some cap on user input to prevent this. However, how do we calculate such a value? It can actually be determined by considering one of the other variables in this problem. If the size of the set is $|S|$, then the size of a powerset is $2^{|S|}$. This large exponential increase needs to be accounted for, as C has an integer limit. `INT_MAX` = 2147483647, the greatest integer that it can take; any higher and the two's complement nature of the bits make that value go into the negatives, which is not what is desired. Therefore if we:

$$\begin{aligned} 2^{|S|} &= 2147483647 \\ \log_2 2^{|S|} &= \log_2 2147483647 \\ |S| &= 31 \end{aligned}$$

Therefore with that size, we will limit the user input to 30 elements in their original set; which makes the time complexity far more forgiving.

1.1.5 Powerset implementations

Now that I had an input, I could finally implement the actual function we had set out to complete initially. Due to my realisation that at least on a higher level, the recursive program shouldn't use that much memory, hence acting as a faster program, I decided to try to implement that first. The problem didn't come from any algorithmic aspects, rather it stemmed from the issue of C and strings. Concatenation is not something that flows seamlessly for C, using the library of `string.h`. This library allows for the function of `strcat`, which combines two strings together, setting the pointer to the first argument. This shouldn't be too much of a problem theoretically, however it rears its ugly head when considering `malloc`. Whenever `malloc` gets involved, so does segmentation faults. What appears to be going on, is that the function is struggling to concatenate the string, as well as output the strings in a meaningful way; with it being very difficult to output syntactic sugar along the lines of braces and commas. Additionally, in C it requires the setting of temp variables to instigate the `strcat`, and therefore it starts to falter on the memory front. Therefore, the other algorithm will be implemented, and that went far more smoothly.

```

1 char **set = getSet(set_size);
2 powerset_size = power(2, set_size);
3 for (index=1; index<powerset_size; index++){
4     for (i=0; i<set_size; i++){
5         if ((index & (1 << i)) > 0){
6             printf("%s", &set[i]);
7         }
8     }
9     printf(" ")
10 }

```

As you can see from the code snippet above, this code follows a nested loop which follows the algorithm described above. The reason this one was so much easier to implement than the recursive one is that the string manipulation doesn't really play a role in the output, rather it only has the string outputted to the screen. Furthermore, C being designed to have lower level bit wise manipulation lends itself perfectly to this method of performing a bit shift.

What is something to note is the use of the power function which is defined in the program, which simply acts as an exponential function, finding the size of the powerset. When considering the size of the powerset, we can then proceed to look at the time complexity of the algorithm. The program loops 2^n times, for each loop it will loop again n times. This sums to produce a time complexity of:

$$O\left(\sum_{k=1}^{2^n} n\right) = O\left(n \sum_{k=1}^{2^n} 1\right) = O(n2^n)$$

Now, having a time complexity that large is not a good thing, as it means that as the size of the set tends toward infinity, it takes an incredibly long time to find the solution, yet I am unsure how to optimize this method to reduce the number of operations. What makes this acceptable is the size limit of sets up to 30, which means that it will max out at that, but as we will see, this can still take an incredibly long amount of time.

Currently, the output isn't the nicest to look at, as the only thing differentiating different elements of the powerset are spaces. We can change this by including braces in our answer, 'syntactic sugar'.

```

1 powerset_size = power(2, set_size);
2 printf("{ }");
3 for (index=1; index<powerset_size; index++){
4     check = 0;
5     printf("{");
6     for (i=0; i<set_size; i++){
7         if ((index & (1 << i)) > 0){
8             if (check > 0){
9                 printf(",");
10            }
11            check = check + 1;
12            printf("%s", &set[i]);
13        }
14    }
15    printf("}");
16 }
17 printf("\n");

```

This program now adds in a few extra details which helps make the program look far better. Firstly, (on line 2) at the start of the program, it will output '{ }' which has the opening brace of the powerset, and the empty set included at the start, as all powersets include the empty set. Having this at the opening not only makes the output accurate, but it also allows us to accept the input size of 0, as the powerset of the empty set is .

Following this (on line 5) we output a comma, and the opening of another brace. The comma is placed in front of the brace so that the final element doesn't have a comma in front of it. This new brace opened will contain the next element of the powerset. Next we have a value named check, which is checking that the first element of this subset has been inputted. By checking this we can then safely append commas (on line 9) in a way that looks natural, separating the individual elements of the subset. Once all elements have been outputted, it will be the end of the first for loop. Henceforth, we can output another closing brace (on line 15). This will repeat until eventually we need a brace to close the powerset, finally giving the solution that we want.

1.2 b.

Write an explanation of your C program. You should incorporate this explanation using detailed code comments and separate written explanations.

1.2.1 Execution

The full code is posted below (without comments), and by following it we can see how and why it is successful.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int power(int x, int y)
5 {
6     int total=1;
7     int i;
8     for (i = 0; i < y; ++i)
9     {
10         total = total * x;
11     }
12     return total;
13 }
14
15 char** getSet(int size)
16 {
17     int i;
18     int j;
19
20     char **arr;
21
22     arr = malloc(size * sizeof(char*));
23     for (int i = 0; i < size; i++){
24         arr[i] = malloc((20) * sizeof(char));
25     }
26
27     printf("Enter %d elements:\n",size);
28     for (i=0; i<size; ++i){
29         scanf("%s",&arr[i]);
30         int found = 0;
31         char* curr = arr[i];
32         for (j=0; j<i; j++){
33             char* temp = arr[j];
34             if (temp == curr){
35                 found = found + 1;
36             }
37         }
38         if (found > 0){
39             printf("There cannot be repeats of elements, enter another one:\n");
40             i = i-1;
41         }
42     }
43
44     return arr;
45 }
46
47 int main()
48 {
49     int set_size;
50     int powerset_size;
51     int index;
52     int i;
53     int check;
54
55     while(1)
56     {
57         printf("Enter the size of the set: ");
58         scanf("%d",&set_size);
59         if (set_size > 30 || set_size < 0){
60             printf("Set size is too large.\n");
61         } else {
62             break;
63         }
64     }
65
66
67     char **set = getSet(set_size);
68
69     powerset_size = power(2, set_size);
70     printf("{}");
71     for (index=1; index<powerset_size; index++){

```

```

72     check = 0;
73     printf("{");
74     for (i=0; i<set_size; i++){
75         if ((index & (1 << i)) > 0){
76             if (check > 0){
77                 printf(",");
78             }
79             check = check + 1;
80             printf("%s", &set[i]);
81         }
82     }
83     printf("}");
84 }
85 printf("}\n");
86 }

```

As we have stuck with so far, we will take the input of $S = \{a, b, c\}$. When we start the program it first takes in a value for the size using scanf, checking whether it is the correct range, and if it is it can proceed. With the size, it calls the getSet function which proceeds to get a set from the user. Instead of having to rely on string manipulation, it takes each line as its own input, its own element. Therefore, there is little room for errors, other than the problem that we use scanf, and so only the first word is gathered from the input. If at any point there is a repeated input, it will report it and the user is able to re-enter the values.

Once the set has been attained, the main function then proceeds to operate on it. A loop is begun from 1 to 2^n , where it then proceeds to loop n times. Each iteration it performs these steps of comparison: Binary form of index AND 00001 shifted i times to the left.

```

1  index = 1, i=0
2  0001 Shift 0 = 0001
3  0001 AND 0001 = 0001 > 0
4  #Is greater than 0, hence will output set[0]
5  Output('a')
6
7  index = 1, i=1
8  0001 Shift 1 = 0010
9  0001 AND 0010 = 0000 = 0
10 #Is not greater than 0, hence will not output anything
11
12 index = 1, i=2
13 0001 Shift 1 = 0100
14 0001 AND 0100 = 0000 = 0
15 #Is not greater than 0, hence will not output anything
16
17 index = 2, i=0
18 0001 shift 0 = 0001
19 0010 AND 0001 = 0000 = 0
20 #Is not greater than 0, hence will not output anything
21
22 index = 2, i=1
23 0001 shift 1 = 0010
24 0010 AND 0010 = 0010 > 0
25 #Is greater than 0, hence will output set[1]
26 Output('b')
27 ...

```

This continues repeatedly until it has occurred for every index, with $n - 1$ bits required. Each time it finishes a subset, it closes a brace; and then at the end it prints a final brace to close the powerset.

For the example, it gives the output of this:

```

Enter the size of the set: 3
Enter 3 elements:
a
b
c
{{},{a},{b},{a,b},{c},{a,c},{b,c},{a,b,c}}

```

Figure 1: Powerset of $\{a, b, c\}$, from the program.

1.2.2 Testing

Since this program is relatively small, there are few areas that need too thorough testing, however there were certain test cases and their outputs need to be explored and identified.

The first test case was the basic $\{a, b, c\}$, which as shown is successful, with the braces in the correct places, same as the commas, same as the actual outputs that came out of it.

Following this, I needed to test the extent of the inputs that worked. This first started with these inputs: Size = 3; Elements: "hello", "456", "(weird"

```

Enter the size of the set: 3
Enter 3 elements:
hello
456
(weird"
{{},{hello},{456},{hello,456},{(weird",{hello,(weird",{456,(weird",{hello,456,(weird"}}

```

Figure 2: Powerset of $\{\text{hello}, 456, (\text{weird"}\}$, from the program.

This is to test that strings of characters work, that numbers work, and that stranger characters such as brackets and quotation marks work, which all of them passed.

What really needed testing though is what would happen if inputs with spaces were taken. Interestingly, it works... in a way I didn't intend nor anticipate. Size = 3 Elements: "hello I am here", "good day", "computer science moment"

```

Enter the size of the set: 3
Enter 3 elements:
hello I am here
{{},{hello},{I},{hello,I},{am},{hello,am},{I,am},{hello,I,am}}

```

Figure 3: Powerset of $\{\text{hello I am here, good day, computer science moment}\}$, from the program.

As we can see... this isn't exactly what was intended. What the program did here is that it took the first 3 words separated by spaces. This is unintentionally a good additional feature to have, as it means elements can be entered another way! What I thought would happen is that the program would instead do the powerset of hello, good and computer. This all stems from the weird behavior of scanf, which I believe is considered one of the greatest mysteries of our time.

Something to test is the empty set case, in which I enter the size to be 0. In this instance the program returns this:

```

Enter the size of the set: 0
Enter 0 elements:
{{}}

```

Figure 4: Powerset of $\{\}$, from the program.

Which is the expected and wanted output.

Similarly, if we choose a value outside the range of 0 and 30, then we get an expected error message, and the user can input the value once more. But, if a non integer input is given, it gives a large spew of errors, for unknown reason. Fortunately there is a simple fix for this, and that is to check if every character in the input is a digit. This is done with an application of the `isdigit()` function in C libraries.

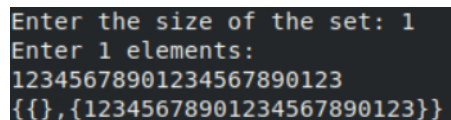
```
1 int intCheck(char s[])
2 {
3     for (int i = 0; s[i] != '\0'; i++)
4     {
5         if (isdigit(s[i]) == 0) {
6             return 0;
7         }
8     }
9     return 1;
10 }
```

And so we rewrite the input program to produce this:

```
1 while(1)
2 {
3     printf("Enter the size of the set: ");
4     char input[5];
5     fgets(input, 5, stdin);
6     input[strcspn(input, "\n")] = 0;
7
8     if (intCheck(input) == 1)
9     {
10        set_size = atoi(input);
11        if ((set_size > 30 || set_size < 0) && intCheck){
12            printf("Set size is not appropriate.\n");
13        } else {
14            break;
15        }
16    }
17 }
```

When you write enough C code, you learn that input validation is the most difficult part.

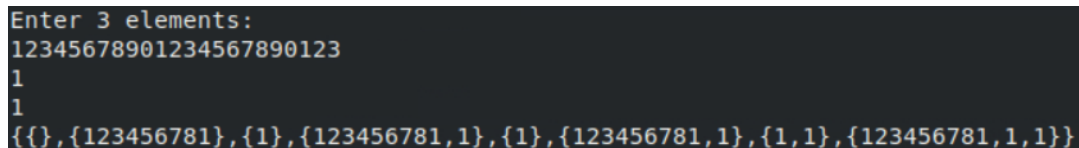
The next area to test is for a single input of "12345678901234567890123", which is just enough characters to be over 20. This is enough that it will exceed the amount allocated in the malloc snippet.



```
Enter the size of the set: 1
Enter 1 elements:
12345678901234567890123
{{},{12345678901234567890123}}
```

Figure 5: Powerset of {12345678901234567890123}, from the program.

When doing this, it works out without failure, even though it exceeds the memory allocated. However, if we then do it with other values.



```
Enter 3 elements:
12345678901234567890123
1
1
{{},{123456781},{1},{123456781,1},{1},{123456781,1},{1,1},{123456781,1,1}}
```

Figure 6: Powerset of {12345678901234567890123,1,1}, from the program.

What we can see here is that the program does not reject elements that are copies now. This is... the weirdness of C and strings. There is little logic or reason as to why this is happening, and whats weirder still is how the string itself has changed, not just cutting off characters, but changing the order entirely. This is one part of the code that

is difficult to optimize without rebuilding the system from the ground up, but to put it simply, C is bad at handling strings.

The final area of testing is how long it will take to execute for larger sizes. This can be done through the time.h library in c, where we can get it to output how long it takes from start to finish. Now initially I wanted to see how long the maximum size array would take, and that is 30. However, upon entering it... it took a long time. Too long for me to be patient. So, the solution is a mathematical one. Since we know the rate at which time increases by n, we can generate use it as a model.

Number of Elements	Average time (seconds)
10	0.000483
12	0.001975
14	0.010293
16	0.046314
18	0.209498
20	0.925608
21	1.902028
22	4.031436
23	8.436948
24	17.45092
25	36.37807
26	76.25563

Using these values, and this model:

$$f(n) = an2^{bn}$$

We can then calculate what a and b are.

$$\begin{aligned}
4.031436 &= 22a \cdot 2^{22b} \\
8.436948 &= 23a \cdot 2^{23b} \\
\Rightarrow \frac{4.031436}{8.436948} &= \frac{22}{23} \cdot 2^{-b} \\
\Rightarrow 0.49955 &= 2^{-b} \\
\Rightarrow \log_2 0.49955 &= -b \\
\Rightarrow -1.00129 &= -b \\
\Rightarrow b &\approx 1 \\
a &= \frac{4.031436}{22 \cdot 2^{22}} \\
\Rightarrow a &\approx 4.3690 \cdot 10^{-8}
\end{aligned}$$

Plugging these two values into our model, we get this equation and these values:

$$f(n) = 4.36895 \cdot x^{2^x}$$

Number of Elements	Average time (seconds)	Expected time (seconds)
10	0.000483	0.000447
12	0.001975	0.002147
14	0.010293	0.010021
16	0.046314	0.045811
18	0.209498	0.206152
20	0.925608	0.916235
21	1.902028	1.924094
22	4.031436	4.031435
23	8.436948	8.429364
24	17.45092	17.59171
25	36.37807	36.64940
26	76.25563	76.23077

To see the graph formed, use the link to desmos below.
<https://www.desmos.com/calculator/yye1jrcm2r>

Using this we can see that the model is actually quite accurate, at the very least to the second, which is really all we care about. At the maximum size the program will take 1407 seconds to run, which translates to roughly 23 minutes and 30 seconds. For sizes larger, which we don't need to consider, I just found it interesting that we can find how many elements it would need for the algorithm to take a day. By using the number of seconds in a day, 86400, we can make use of the Lambert W function to find the value of x that associates with it.

$$\begin{aligned}86400 &= 4.36895 \cdot 10^{-8} \cdot x 2^x \\ \implies 1.9775 \cdot 10^{12} &= x e^{x \ln 2} \\ \implies 1.9775 \cdot \ln 2 \cdot 10^{12} &= x \ln 2 \cdot e^{x \ln 2} \\ \implies W(1.3707 \cdot 10^{12}) &= W(x \ln 2 \cdot e^{x \ln 2}) \\ \implies 24.73804 &= x \ln 2 \\ \implies 35.689 &= x \\ \implies x &\approx 36\end{aligned}$$

This means if 36 elements were an acceptable input, it would take a whole day to output the full powerset.

1.2.3 Conclusion

To conclude, the powerset algorithm made has several strengths, but also a multitude of weaknesses. In areas that it succeeds is that it manages to complete the task for nearly any input. By making use of strings, it means that we can find the powerset of any set, not just ones consisting of numerical values. Furthermore, the program allows for the output to carry set notation, and follow appropriate, expected syntax. Finally the algorithm used lends itself to C, carrying out the low level bitwise operations.

However, there are several flaws with the program, mostly in its limitations. Firstly, the algorithm to produce the powerset is unlikely to be optimized, as we can see from the incredibly inefficient time complexity, and the fact that going up to even just 36 elements takes over a day to compute, which is entirely unrealistic to use. That leads to another issue though, and that is the limitation on the number of elements, due to the limits of C with its maximum value of the integer data type. Additionally we need to consider how the program handles erroneous inputs, as whilst it tries to tackle most cases, there are exceptions that don't make sense. So the majority of cases that are handled removes inappropriate inputs for size of the set, and ensuring it is actually a number. If there was one area to change or improve drastically, it would be using a different method to get inputs, rather than the incredibly broken `scanf`, to instead use the less broken `fgets`, which we will see a lot more of in the second question. Therefore, whilst the program has areas in which it fails, for near all cases - in which the user isn't actively trying to break the program - it is an effective program.

2 Question 2.

The C programming language allows for us to easily work with files. In this question you will implement a command-line editor that is capable of creating, displaying and manipulating text files. You have a lot of freedom in how you do this but you will need to make informed design decisions and provide justifications for these choices as part of your documentation.

2.1 a.

Write a C program that allows a user to perform each of the operations listed below on text files. You must provide a command-line interface and operate on files in your current working directory but, apart from these requirements, you may implement the program in any way. You should document all design decisions not covered above, e.g., how a user specifies the operation they want to perform.

2.1.1.1 Decomposition

When faced with a problem of this scale and magnitude, we need to break it down into individual smaller problems so that it even seems manageable. Fortunately, this question has already been done most of the hard work here. The problem has already been separated into 10 different functions that we need to implement. What we need to discern though is what parameters they take, what they should do, and what they should return. Furthermore, we need to figure out how we are going to be taking these inputs, and what the user interface will look like.

Breaking down the functions given though are hard to do without some preliminary knowledge. C fortunately has powerful tools for manipulating and altering text files, we just need to figure out how to harness them.

2.1.1.1.1 Preliminary knowledge

Through researching through a multitude of online sources (found in the bibliography), as well as the labs there are a few basics that must be understood to mess with files. The first thing to do is to open file, and this is done through this snippet of code:

```
1 FILE *fp;
2 fp = fopen(name, mode);
3 if (fp == NULL){
4     printf("File not found.\n");
5     return 0;
6 }
7 fclose(fp);
```

This snippet does 3 things. Firstly, it defines a file pointer, `fp`. This is used to make reference to the file and the location within the file. Following this we set the pointer equal to `fopen(name, mode)`. Here, the name is the name of the text file in the current directory, and the mode is what is intended to be done to the file, this includes "r" for read, "a" for append, etcetera. The next thing it does is to check that there has been no issue in making the file/finding the file. If there has been, the pointer will be defined as `NULL`. Finally, the code closes the file, and this is so that the memory allocated to it by the program, is freed.

What is important to note here, is that the `fopen` function acts in two ways. In the event that the text file specified in the name parameter exists, then that file will be selected and operated on. However, if no file has the name specified, then it will create a file, rather than returning an error. This will be important later.

The only other functions that is essential to know for nearly all functions is the `fgetc(fp)` and `fprintf(fp, "%c", c)`. The first function takes the file pointer, and returns a character that the pointer is currently pointing to. Then once it has returned it, it increments the pointer. This function is so powerful as if you repeatedly call this function `n` times, then it will get the `n`th value in the text file. This will form the basis of all of the future functions to implement. The second function does the opposite. It takes the file pointer, and appends a character to it. With these, we can input into text files, and take outputs from them. However, there is a catch. The limitation is found in manipulating the file pointer. Finding a way to manipulate the pointer so it is in the correct location when desired is the most difficult part of these functions, and will be the main fundamental area of algorithmic thinking.

2.1.1.1.2 Function details

Now we can break down the functions at hand, and the details necessary for designing them.

One thing to note before figuring this out, is that the return value for any function should be 0, as it a series of operations - a process - not a function as you will.

createFile():

- Parameters:
 - `char* name`: name of text file
- Processes:
 1. Create an empty file with the name specified.

copyFile():

- Parameters:
 - `char* copying`: name of text file being copied
 - `char* name`: name of the new text file

- Processes:
 1. Create an empty file with the name specified.
 2. Copies content from the original file to new file.

deleteFile():

- Parameters:
 - char* name: name of text file
- Processes:
 1. Delete text file specified.

showFile():

- Parameters:
 - char* name: name of text file
- Processes:
 1. Find the content in the text file specified.
 2. Output the content to the user.

appendLine():

- Parameters:
 - char* name: name of text file
 - char* text: input that user wants to append
- Processes:
 1. Traverse to the end of the text file specified.
 2. Add the input as a line of text.

deleteLine():

- Parameters:
 - char* name: name of text file
 - int line: line of text user wants deleted
- Processes:
 1. Create an empty file with the name of the original.
 2. Copy each line of content from the original until the specified line is reached.
 3. Iterate through the line, then copy the rest of the file.
 4. Delete the original file.

insertLine():

- Parameters:
 - char* name: name of text file
 - int line: line that the user wants to insert text
 - char* text: input that user wants to insert
- Processes:
 1. Create an empty file with the name of the original.
 2. Copy each line of content from the original until the specified line is reached.
 3. Append the string inputted for that line.
 4. Copy the rest of the file.
 5. Delete the original file.

showLine():

- Parameters:
 - char* name: name of text file
 - int line: line of text user wants shown
- Processes:
 1. Traverse to the line of the text file specified.
 2. Traverse through the line, and output the string.

showLineNum():

- Parameters:
 - `char* name`: name of text file
- Processes:
 1. Traverse to the end of each line, and increment a counter.
 2. Output the counter.

Now there is a function exempt from this, and that is the changelog functions. This is because that is the area of work that is best suited to the end, and so all details regarding that function will be last.

2.1.2 Design

2.1.2.1 Create File

Out of all of the functions to make, this one is fortunately the simplest. If we recall that the `fopen` function if it cannot find the file will create a file with the name specified. So the function should just be as simple as copying the code from above. However, there is slightly more depth to this.

What we need to consider when writing this function, is that we don't want to continually repeat this code snippet of opening the file, testing if it exists, returning an error message if it doesn't etc. So the solution to this is to define this series within a function. My initially function for this was in essence a direct clone of the code above:

```
1 FILE * declareFile(char* name, char* c)
2 {
3     FILE *fp;
4     fp = fopen(name, c);
5     if (fp == NULL){
6         printf("File not found.\n");
7         return 0;
8     }
9     return fp;
10 }
```

This should have worked, but I didn't considering what was being returned. As most of these functions are actually more processes (not returning meaningful values, rather just executing operations), it doesn't mean much to return 0 in this function. The reason for this is because as this function will be contained within all other functions, and at no point will we write 'return declareFile()', which means that if it does error, it will carry out the operations below it until it errors. Therefore my solution was to have the `declareFile` function be limited to this:

```
1 FILE * declareFile(char* name, char* c)
2 {
3     FILE *fp;
4     fp = fopen(name, c);
5
6     return fp;
7 }
```

This lets me do two things, it saves a couple lines of code from declaring and defining a file every two seconds, but more importantly it keeps this area separated from the rest, it improves the modularity of the code. Additionally, the exclusion of the if statement actually allows us to have more control over what is outputted to the user. For example if the program cannot create a file, that is a different problem to the program being unable to find the file. This fix lets me improve debugging, and also lets the error messages for the user be more useful.

Hence, we can finally design the `createFile()` program.

```
1 int createFile(char *name)
2 {
3     FILE * fp = declareFile(name, "w");
4     if (fp == NULL){
5         printf("File not found.\n");
6         return 0;
7     }
8     fclose(fp);
9     return 0;
10 }
```

The reason that the `fclose` follows afterward, rather than in an else statement is for no particular reason. The main thing of importance, is that an error is returned if `fclose(NULL)` is called. This avoids this problem.

2.1.2.2 Delete File

This was the next command I implemented, as I needed this command to go and design future commands. This is because for insertLine() and deleteLine(), the need to delete a file is required. Fortunately, this is likely the easiest command I will have to implement. This is because the C library has a function of: remove(name). I wish there was more depth to this but, why reinvent the wheel. The function of remove, will just be used in future.

2.1.2.3 Show File

This next command introduces how to access and pull from files. First we open the file in the read mode, specified with "r". Once we have the file pointer (fp), we access the character using the fgetc(fp) function. Then it will output the character to the user using printf. Then when fgetc is called again, it updates the pointer to the next character. This will repeat until it reaches the end of the file which we can determine using "EOF", a character built into the language we can use to refer to the end of a text file. We don't need to include any details surrounding newlines as the text files have newline characters built into the language. Hence, we can finally make the function.

```
1 int showFile(char *name)
2 {
3     char c;
4
5     FILE * fp = declareFile(name, "r");
6     if (fp == NULL){
7         printf("File not found.\n");
8         return 0;
9     }
10
11     while(1)
12     {
13         c = fgetc(fp);
14         if(c==EOF){
15             break;
16         }
17         else{
18             printf("%c", c);
19         }
20
21         fclose(fp);
22         return 0;
23 }
```

What is important to note, is that the reason the printf statement is contained in the if statement, is because if we were to printf before we break, then it will try to output the EOF character. Now this isn't an inherent problem if it were to act like a newline or terminating character, but instead C interprets it as ª or in numerical characters a -1, which especially is a problem in later programs when writing between certain files.

2.1.2.4 Copy File

Of the file manipulation commands, this one is the hardest to implement, yet it is just an application of everything designed so far. Here, we first declare the two files. We want to read from the first file, and we want to append to the second file. Defining the first file as fp1, and the second as fp2 becomes a recurring naming system. To save time, we won't actually use the createFile function from earlier, as that does not open the file we want it too; and we would have to reopen the file as the createFile code closes it. We open the file withNow, we do exactly what we did in showFile function, yet replace printf with fprintf. If we recall the parameters of fprintf; we enter the file we are writing too, the data type, and the variable being added, in this case a character.

```
1 int copyFile(char *copying, char *name)
2 {
3     FILE * fp1 = declareFile(copying, "r");
4     if (fp1 == NULL){
5         printf("File not found.\n");
6         return 0;
7     }
8
9     FILE * fp2 = declareFile(name, "a");
10    if (fp2 == NULL){
11        printf("File not found.\n");
12    }
```

```

12     return 0;
13 }
14
15 char c;
16 while(1)
17 {
18     c = fgetc(fp1);
19     if(c == EOF){
20         break;
21     }
22     else
23         fprintf(fp2, "%c", c);
24 }
25
26 fclose(fp1);
27 fclose(fp2);
28
29 return 0;
30 }

```

Another thing to make note of, is why this function was implemented like this, instead of doing a series of `appendLine` functions which theoretically would be less steps, and greater modularity? The reason for this is because it ignores the longest step of them all; opening and closing the file. In `appendLine`, it would need to open the file and close it, doing that `n` times over, where `n` is the number of lines. Additionally the function would have to isolate each line of code. This program does that all in one. What is lost in reusing code, makes for an optimisation in what the function is trying to achieve.

2.1.2.5 Append Line

Now we are tackling the line functions and we are given a far more interesting problems to deal with. Starting with this, we reapply what we used with the `copyFile`, where we set the file pointer to append. The reason we do this is because it sets the file pointer to the end of the text file, which means we don't need to needlessly cycle through the whole text file just to reach the EOF. Once the pointer is at the end of a file, using `fprintf` on the pointer will automatically position the next character on to that line, and will continue to do so until a different line is appended, or a newline character is encountered.

With the two parameters of the name of the file and the actual text to be appended, once the file pointer is defined, we will want to iterate until we reach the end of the string, and each iteration append a character from the string. If we recall the `fprintf` function though, we can see that it can append strings as data types can be selected. However, the reason I am avoiding that is because not only is working with just characters more consistent in the writing of my code, but also it means that the weirdness of C and strings cannot touch this function. Throughout this question, there is a focus and emphasis on modularity, which can be seen by the efforts made already to isolate these functions. The data entered into these functions are intended to be pure, inappropriate inputs filtered out. Hence, by working in characters ensures that there is no risk of whatever data entered producing an incorrect output from different definitions of input functions and strings from within C.

The only other area to touch on, is that we need to add a new line on to the end of the document. The reason for this being after the program, rather than before, is simply an observational choice. When using the text files on the system it reads the line numbers down the side, and it reads one up from where there is a new text file. That, and if I added a new line at the beginning, it creates a strange gap. This is because when using the mode `append`, it seems to immediately go to the next line.

```

1 int appendLine(char *name, char *text)
2 {
3     int i = 0;
4     FILE * fp = declareFile(name, "a");
5     if (fp == NULL){
6         printf("File not found.\n");
7         return 0;
8     }
9
10    for (i=0; i< strlen(text); i++){
11        fprintf(fp, "%c", text[i]);
12    }
13 }

```

```

14 fprintf(fp, "%c", 10);
15 fclose(fp);
16
17 return 0;
18 }

```

What can be seen is that when adding a newline at the end, instead of the newline character '\n' being referred too, it uses the number 10. They are equivalent, and due to some weird reason, more string functions worked using that instead of that newline character. Whenever 10 is used, it is actually signifying a newline.

2.1.2.6 Show Line

This next function is a gentle introduction into the line manipulation I will need to be doing. The process is similar to previous ones, in which it must traverse through the text file, however the difference is that it must track the line. This uses an integer counter that increments whenever a newline character is encountered, referred to by 10. My original code followed as such:

```

1 while(1)
2 {
3     c = fgetc(fp);
4
5     if (1 == line){
6         printf("%c", c);
7     }
8
9     if (c == 10){
10        l = l + 1;
11    } else if (c == EOF){
12        break;
13    }
14 }

```

This should work, and it does, aside from the issue of the EOF character. The EOF character rears its ugly head when calling for the line one greater than the number of lines in the file.

If we look at the example text file below, we can see the contents of Textfile.txt.

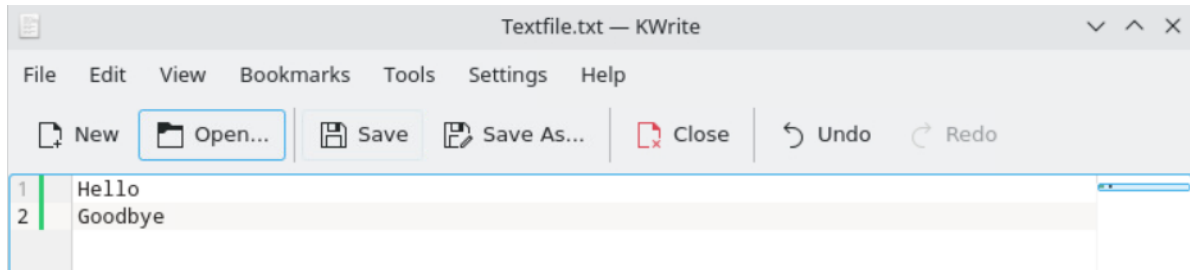


Figure 7: Screenshot of the contents of the text file.

If we plug values into the function we get these outputs:

```

showLine(Textfile.txt, 1) = Hello
showLine(Textfile.txt, 2) = Goodbye
showLine(Textfile.txt, 3) = ¨

```

So, this function needs to be rewritten so this isn't the case. The way we do this is by rearranging the if statements.

```

1 int showLine(char *name, int line)
2 {
3     int l = 1;
4     char c;
5
6     FILE * fp = declareFile(name, "r");
7     if (fp == NULL){
8         printf("File not found.\n");
9     }
10 }

```

```

9     return 0;
10 }
11
12 while(1)
13 {
14     c = fgetc(fp);
15
16     if (c == EOF){
17         break;
18     } else {
19         if (l == line){
20             printf("%c", c);
21         }
22
23         if (c == 10){
24             l = l + 1;
25         }
26     }
27 }
28
29 fclose(fp);
30 }

```

This small change puts the checking of EOF a priority in the program, and hence it will always break out of the while loop upon encountering that character. Whilst this isn't as elegant, it is the best solution I could find to the problem.

2.1.2.7 Delete Line

This is the first of two functions that brought me much greater difficulty and depth. The reason for this is because there are two valid ways of completing this, one more arduous than the other. The method that would likely make the most sense to people, is to traverse the text file until the pointer is in the correct location, then use some method to enter the correct characters. This way, it would only require finding the location, getting to it, and then changing values, before being complete. However, when I first began to implement this function, I could not find a function that would allow me to write to the file without wiping the whole file entirely. This lead me down another path of solutions then.

My idea was to open the original text file, then create a new text file, copy the contents of the current text file over, then ignore the text on the specified line, finally appending over the other text. Then, we would use a rename function that is built into C to rename the file back to the original text file, whilst deleting the original file.

```

1 int deleteLine(char *name, int line)
2 {
3     int l = 1;
4     char c;
5
6     FILE * fp1 = declareFile(name, "r");
7     if (fp1 == NULL){
8         printf("File not found.\n");
9         return 0;
10    }
11
12    char temp[100] = "TemporaryHolder.txt";
13    FILE * fp2 = declareFile(temp, "a");
14    if (fp2 == NULL){
15        printf("File not found.\n");
16        return 0;
17    }
18
19    while(1)
20    {
21        c = fgetc(fp1);
22        if (l == line){
23            if (c == 10){
24                l = l + 1;
25                fprintf(fp2, "%c", 10);
26            }
27        } else {

```

```

28     if (c == 10){
29         l = l + 1;
30     } else if(c == EOF){
31         break;
32     }
33     fprintf(fp2, "%c", c);
34 }
35
36 }
37
38 deleteFile(name);
39 renameFile(temp, name);
40
41 fclose(fp1);
42 fclose(fp2);
43
44 return 0;
45 }

```

The way the algorithm works is by keeping a line counter, updating whenever it encounters a newline character. Then, when the counter is equal to the specified line, it adds a newline to the file and increments the line, effectively jumping that line. As we can see, the function requires there to be a unique name for the temporary file, since the OS cannot handle two of the same name, hence why the temp name may not stick, since it still needs work to ensure no clone name.

This was my approach for both this and the insertLine operations, but it soon came to my attention that there is a way to manipulate the text file without wiping the whole file, it just requires that it is opened with mode r+, and the function putc is used.

The function putc() takes two parameters: the character to add, and the file pointer. If we were to do code like this:

```

1 char c;
2 c = fgetc(fp1);
3 c = fgetc(fp1);
4 c = fgetc(fp1);
5 putc('2', fp1);

```

This will then make the 4th character of the text file be 2 as well as incrementing the file pointer once more. With this function, and fseek() which allows for the file pointer to be set the start, the end, or any offset of the current, you would then think this could be utilized to go and rewrite the algorithm so that it will put 0 characters every single character that is on that line. However, there seems to be a difficulty that I cannot overcome, and that is ensuring that the program comes to an end. Writing a similar program to before, where it correctly iterates through the file we get this code:

```

1 int deleteLine(char* name, int line)
2 {
3     FILE * fp1 = declareFile(name, "r+");
4     if (fp1 == NULL){
5         printf("File not found.\n");
6         return 0;
7     }
8
9     int l=1;
10    char c;
11
12    while(1)
13    {
14        c = fgetc(fp1);
15        if (c != EOF){
16            if (c == 10){
17                l = l + 1;
18            } else {
19                if (l == line){
20                    putc(0, fp1);
21                    fseek(fp1, -1, SEEK_CUR);
22                }
23            }

```

```

24     } else {
25         break;
26     }
27 }
28 fclose(fp1);
29 return 0;
30 }

```

This code... does not work. The reason for this is that when it gets to the line specified, it doesn't seem to recognise the newline character, and will add characters indefinitely, not only having the program never close, but also having the rest of the text file wiped. I have tried a large combination of rearranging the values and if statements, and the problem seems to lie in a lack of understanding of `putc`. I say this because if I replace `putc` with `printf`, outputting the character, it acts completely as expected. Hence, for the time being I am going to stick with the original solution, since it works quite effectively, especially since when the file is deleted it isn't stored in wastebin, therefore there isn't an absolute waste of space. However, it does have two major limitations currently.

Firstly, if the file is named the same as the temporary name holder, then it will likely cause an error within or outside of the terminal. The solution to this is to find a name of a file that the user will never be able to input, or a name of a file that will be distinct from the input name. This area may need work.

Secondly, if the computer is lacking space, then perhaps there will be a failure as it will not be able to hold both text files simultaneously. This would not happen in the case of the small scale programming and text files that we are operating with, however if used at much larger and grand scales, it is very possible that the effective temporary copying of a large file will result in running out of space, and so modifying a currently existing file is far more preferable than creating an entirely new file. These two problems hold for the next function to be discussed, as it requires a similar technique.

2.1.2.8 Insert Line

Being the function with the greatest number of parameters, this one needs the most algorithmic thinking. Firstly, I needed to make the decision of what insert would actually do. Inserting a line seems intuitively obvious, but the question is if it should replace the content on that line. My conclusion was to not, but rather shift all of the exiting content down a line. By doing this it means replacing a line is entirely doable, as it will just entail deleting a line, and then inserting on that line.

Going off of the previous function, we will create a new file that will copy all of the data from the original file, inserting the new line of data into it. This can be broken down into five distinct stages:

1. Open the original text file, and create a new text file with a temporary name.
2. Copy the contents of the original file into the new file, up until reaching the line to insert in.
3. Append the line of text specified as a parameter.
4. Copy the rest of the content from the original text file.
5. Delete the original text file, and rename the new file.

By using another text file, this function shares similar problems with the `deleteLine` function, however the use of it is far more useful here as it gives a far easier solution. This is because the data is being shifted, if we were to amend and alter the initial file, then all characters past the line specified would also need to be shifted by the number of characters in the input. However, for the new text file method, it means that we only need to insert the line, then append the rest of the text - the shift occurs naturally allegorically.

There is another case we need to consider, and that is that the user is trying to insert a line that doesn't exist yet. For example if a text file only has 5 lines, and the user enters the line to be 7, then what should the program do. The simpler solution would be to just ignore it, and say anything greater than the number of lines isn't allowed. However, if I were using a text editor, I would want to be able to insert lines much later on with a gap. Hence, the solution is that if when the program is copying content over, before it has reached the specified line and it reaches the end of the file, it will add a new line until it reaches the specified line. This way it keeps the input entirely valid.

```

1 int insertLine(char *name, int line, char *text)
2 {
3     int l = 1;
4     int i;
5     char c;

```

```

6
7 FILE * fp1 = declareFile(name, "r");
8 if (fp1 == NULL){
9     printf("File not found.\n");
10    return 0;
11 }
12
13 char temp[100] = "TemporaryHolder.txt";
14 createFile(temp);
15 FILE * fp2 = declareFile(temp, "a");
16 if (fp2 == NULL){
17     printf("File not found.\n");
18     return 0;
19 }
20
21 while(1)
22 {
23     if (l == line){
24         break;
25     } else {
26         c = fgetc(fp1);
27
28         if (c == 10){
29             l = l + 1;
30             fprintf(fp2, "%c", c);
31         } else if (c == EOF) {
32             l = l + 1;
33             fprintf(fp2, "%c", 10);
34         } else {
35             fprintf(fp2, "%c", c);
36         }
37     }
38 }
39
40 for (i=0; i< strlen(text); i++){
41     fprintf(fp2, "%c", text[i]);
42 }
43
44 while(1)
45 {
46     c = fgetc(fp1);
47     if (c == EOF){
48         break;
49     } else {
50         fprintf(fp2, "%c", c);
51     }
52 }
53
54 deleteFile(name);
55 renameFile(temp, name);
56
57 fclose(fp1);
58 fclose(fp2);
59
60 return 0;
61 }

```

Here we can see the program broken up into the steps described. The first while loops in the second step, in which it is copying across the data. Here it has to make 2 checks of whether it is a newline character or an end of file character. As specified, if it is the EOF character, then we will want to add a new line to the new text file, done using `fprintf(fp2, '%c', 10)`, which will go on until the line counter equals line. Following this is appending the inserted line of text to the file, and then proceeding to copy the text as usual. This program to insert a line feels more successful, in that the efficiency of the program is much more tied to the method it is used, and the faults in the method are made up for by that.

2.1.2.9 Number of lines

As opposed to the previous couple functions, this one is far easier to figure out. By traversing through a text file a character at a time, every time a newline character is encountered it increments a counter by 1, and then the loop ends when encountering the EOF character. Therefore we can string together some fairly rudimentary code to fulfil this. However, there are two things to consider.

Firstly, we have to decide the number of lines are in an empty file, 0 or 1. This is important as this will be what our counter starts at. The solution for this comes from the FAQ, in which it discusses what details the changelog should have, the number of lines in a newly created file being 0. Hence we will set the counter to start at 0. I was still worried about there being a potential issue though, and that is in regards to manually typing out a text file, and ensuring it only has one line, as would that not entail that it would have 0 lines, even though one line has text? Fortunately, the way the text files seem to be constructed is that before the EOF character, there is a newline character, and so when experimenting in code, it still works.

Secondly, we may actually change the standard created so far. All previous functions acted as processes, with no important or relevant return value, however it holds to reason that numLines will actually be useful in other functions, most notably the changelog construction, and potentially in the later additional functions. Despite, this breaking the standard of the program, it can still be used as effectively, as it is just returning an integer. With those requirements set out, the function can be designed like so.

```
1 int numLines(char * name)
2 {
3     char c;
4     int l = 0;
5     FILE * fp = declareFile(name, "r");
6     if (fp == NULL){
7         return 0;
8     }
9
10    while(1)
11    {
12        c = fgetc(fp);
13        if (c == 10){
14            l = l + 1;
15        } else if (c == EOF){
16            break;
17        }
18    }
19
20    return l;
21 }
```

2.1.3 User input

From the design section, we can see two major areas being left out, and that is the user input and changelog. The changelog functions are difficult to implement without a basis of how the inputs are going to go. Additionally, inputs and validation are usually the most difficult part of any C program, and so by getting it out of the way and designing it effectively, it will heighten the rest of the project.

2.1.3.1 Method of input

Initially, I had to decide on the method of which I would gather user input. There were two major viable options, one far easier than the other. The first method would be to have a menu system. This would be where you select a command from a menu, then proceed to input the parameters one by one, there being a separate input section for each. This option bares similarity to how question 1 was answered, where first the integer size of the set had to be given, and then separately a series of other string inputs once that was chosen. This method is far easier to implement and less prone to errors as it means that data types can be used appropriately, as each section knows what input it wants. The other option, has much more ambiguity.

The alternative is to replicate command lines and terminals that we use every day. This would mean that each input would specify the command, and then the following parameters of the function after it. This adds difficulty as it requires a lot of string manipulation, as well as precision in ensuring that the inputs gathered are exactly what we want. If it is successful though, then we will have a far more efficient system to enter commands, without the

tedium of entering each part individually. Furthermore, I feel as if I must pursue this method, as to gain a better understanding and grasp of the way C handles strings. Hence, this harder yet more gratifying method is the one that is chosen, and with it comes planning and complications.

2.1.3.2 Scanf vs fgets

Up until this point I had been exclusively using scanf, unaware that - in Yoda's words - there is another. The way scanf works is that it specifies a certain data type, and then the variable that it is storing the user input in. However, this only takes the characters of the input up to the first white space, meaning that any spaces in the input, effectively have the remainder of the string ignored. This is very inconvenient to our plan and current design, so it needs a change. Introducing the cooler older sibling of scanf is fgets, which takes in three arguments: the string to store too, the number of characters to take, and the pointer, which in our case will always be stdin. This should bare some semblance to the fgetc command, which it does as fgets can also be used to retrieve strings from files. This function will continue up until the newline character, which makes it incredibly useful in gathering whole strings. However, it is too good and will also take the newline character as a part of it, which in some instances is what we want, in most it only inconveniences. Thankfully using some functions from the library string.h we can quite easily remove that character. The snippet of code below describes using the code in action, and how to remove that character.

```
1 char input[100];
2 fgets(input, 100, stdin);
3 input[strcspn(input, "\n")] = 0;
```

Here a string is defined, called input, holding 100 characters (may change in future). Then fgets takes whatever the user input is and places it inside of input. Then the strcspn function is called, which takes a string and a substring of what it wants to find; the function returns the index of where that substring is in the string. Using this, we then set the newline character in input to 0, ensuring no strange characters at the end.

2.1.3.3 Interpreting the input

Now that we have a string that contains the command and all of the parameters, before we start validating it, we need to know how to isolate each of the parameters, and even how we want to isolate them. The most convenient method would entail that the parameters would be split into a two dimensional array, each element containing a string. But for this method to be effective, I would have to be able to determine where each parameter begins and ends, and asking for specific notation from the user makes it ever more inconvenient. Although, it felt as if this design was falling apart, there were solutions I thought of that let me progress to the next part.

Firstly, the standardised ordering of parameters was important. If we look at all of the functions that are designed, they range between one and three parameters of three different types: names of a text file, line numbers and text values to be added to a text file. The problem at hand is that if spaces are used to distinguish between parameters, then that would limit text to have no spaces. The fix for this though, is to always have text be the final parameter. For example, if we look at appendLine, it takes in the name of the text file, and it takes in a entry of text; so then if we define everything after the first space to be the text string, then it means that spaces can act as our separation of values. The syntax for example would be as such:

```
>>> appendLine Textfile.txt This is the text to add.
```

Or another example:

```
>>> insertLine 5 Textfile.txt Inserting this text here on to line 5.
```

There is an unfortunate trade off though, and that is that it is unable to accept files with names with spaces in them. This isn't ideal, but it enables for much greater and cleaner parameter inputting, therefore it is a sacrifice worth making (who names files with spaces in them anyway?).

Now the formatting is set, we actually need to find a way to implement that formatting. Fortunately, there is a function in string.h, which perfectly suits our needs. This function is strtok, which takes in two parameters: a string, and a delimiter. The delimiter is the character that is what the string should be separated by, so for our case, the delimiter is a whitespace.

```
1 char delim[] = " ";
2 char *command = strtok(input, delim);
```

```

3 char* parameter1 = strtok(NULL, delim);
4 char* parameter2 = strtok(NULL, delim);

```

In the code snippet above, this will separate the string into three, where the whitespaces are removed. For example if the input was:

```
>>> deleteLine Textfile.txt 3
```

Would give that:

```
command = "deleteLine"; parameter1 = "Textfile.txt"; parameter2 = "3"
```

I would be lying if I said that I understood how and why this works, especially with the NULL values, but all that is important to understand is that this separates the values, and that if there are no characters after another space, then it returns NULL (this will be important for later).

If we wanted to claim the rest of the text, then we simply make reference to the existing pointer in NULL, and set the delimiter to be no character.

```

1 char delim[] = " ";
2 char *command = strtok(input, delim);
3 char* parameter1 = strtok(NULL, delim);
4 char* parameter2 = strtok(NULL, "");

```

```
>>> appendLine Textfile.txt This text is to be inputted.
```

Using this code snippet and this input, we would get:

```
command = "appendLine"; parameter1 = "Textfile.txt"; parameter2 = "This text is to be inputted"
```

Now that we have the fundamental building blocks, we need to string them together in an effective way. We need a function that will isolate the parameters and will return an array of those parameters, this way we can reduce the number of pieces of repeated code. The difficulty here is surrounding the number of different permutations of entry there is per command. Furthermore, if there is an error we want to be able to tell the user what the error is; and let the main program know not to go ahead with the function. This adds complications as the function is already returning an array, so we can't return some other value. At first I was going to introduce a check variable and pass its address into the function; but then I realised a far more powerful tool in the form of utilizing NULL. NULL can be returned from any function, and so if an error stems, we can return this, and the main will know not to operate the function.

```

1 char **getParam(char* input, int paramNum, int text)
2 {
3     char **arr;
4     char delim[] = " ";
5     arr = malloc(paramNum * sizeof(char*));
6     for (int i = 0; i < paramNum; i++){
7         arr[i] = malloc((100) * sizeof(char));
8     }
9     for (int j = 0; j < paramNum-text; j++){
10        char* temp = strtok(NULL, delim);
11        if (temp == NULL){
12            printf("Not enough parameters.\n");
13            return NULL;
14        } else {
15            arr[j] = temp;
16        }
17    }
18
19    if (text == 1){
20        char* temp = strtok(NULL, delim);
21        if (temp == NULL){
22            printf("Not enough parameters.\n");
23            return NULL;
24        } else {
25            arr[paramNum-1] = temp;
26            arr[paramNum-1][strlen(temp)] = ' ';
27            return arr;

```

```

28     }
29 } else {
30     char* paramCheck = strtok(NULL, delim);
31     if (paramCheck == NULL){
32         return arr;
33     } else {
34         printf("Too many parameters.\n");
35         return NULL;
36     }
37 }
38 }

```

This function can be broken down into smaller chunks. Firstly we go about defining the 2D array using malloc, having the same number of elements as the number of parameters. This is done so that we can ensure the correct allocation of size to each of the individual parameters. Once the array is defined, we then proceed to loop the same number of times to assign each of the parameters a value. If at any point the the strtok function returns NULL at this point, it means that there aren't enough parameters for the function, and therefore it returns that error and NULL accordingly.

This isn't fully accurate, as actually the number of times it loops doesn't entirely depend on the number of parameters (paramNum), but also a checker value of text. If text = 1, then the number of times it will loop will be subtracted by 1, as this means that the final parameter is a text parameter.

This brings me to the next segment of code, which focuses on the what the final parameter is. If text = 1, then it will check that there is text to append to the final array (if not it will return the not enough parameters error). If there is, then it will set the final parameter to the text, using strtok as described. If text = 0, then it is checking if there are too many parameters. It does this by performing another strtok, and checking if it is NULL. If it is not NULL, it suggests there is another parameter after what should be the last one, and so it returns the error that there are too many parameters.

This method of gathering the user's input is successful, as it helps to isolate the parameters required, returning them in a way that is useful to us.

2.1.3.4 Commands and functions

Now we have a way to gather and isolate the parameters, we now want to be able to differentiate between commands and pass in the correct parameters in each. Firstly, we will lock every function ahead into a while loop, so that multiple operations can be performed until the user closes it. Since we are going in for a style of a command line, we will start every input with '!!!' (potentially including the current directory before it later down the line). Secondly, we will ask for the user's input using the fgets code from earlier. Next, we will retrieve the first token from the input which should be the command. From here, we will unfortunately want to do the disgusting set of 10 if statements for each command, specifying what it should do afterwards.

Each if statement will follow a similar structure

1. Checks whether the command is a certain string.
2. Sets an array of parameters using the getParam function.
3. Checks if the function returns null.
4. If not, then call the function with the correct parameters.

The best way to demonstrate this is with an example, in this instance being the createFile function.

```

1  if (strcmp(command,"crt")==0 || strcmp(command,"create")==0){ /* create: 1 parameter */
2      char** parameters = getParam(input, 1, 0);
3      if (parameters != NULL){
4          createFile(parameters[0]);
5      }
6  }

```

Here we can see that the strcmp from string.h is used, which is a function which compares two strings. If the strings are equal it returns 0. So, to identify the command of createFile, I give two possible command words. Either 'create', or 'crt'. Since for other functions it can take a while to write out the full word, it seems to make

sense to have a shorthand version as well as the full one. It was also to give some uniqueness and identity to my program. Next we can see that a list of parameters is gathered with the arguments of the input string, the number of parameters (1), and a 0 to signify there is no text variable.

Now if we apply this on a larger scale, we can get the full code, but there are a couple other extra points to tackle. In regards to the functions that take line inputs, what is required is that the line input is actually appropriate. For this, we will want to be using the `intCheck` function discussed in question 1, ensuring that the string is only comprised of digits. Then we can use the inbuilt function of `atoi` to convert that into a variable of the integer type.

Additionally, there are some other operations we will want to implement for ease of use. To specify these aren't the additional features for question b. rather just functions that the user - in particular myself - would find useful in navigating the the text editor. First of these is `rename`, or `rnm`, which is implemented as a user function as it fitted (since it was already implemented). Next is `exit`, or `ext`, which breaks from the while loop and closes the program (an alternative to `Ctrl C`). Finally there is `manual`, or `man`, which outputs text that divulges all of the existing commands, and how to use them:

```
Commands take parameters separated by spaces; aside from text which can continue indefinitely.
The list of commands and parameters are below:
- rnm targetFileName newFileName : renames the specified file, with the newly specified name.
- crt fileName : creates a text file with the name specified.
- cpy targetFileName newFileName : copies the specified file, giving it the newly specified name.
- del targetFileName : deletes the file with the name specified.
- shw targetFileName : shows the contents of the file with the name specified.
- appln targetFileName text : appends a new line of text to the specified file.
- delln targetFileName line : deletes a specified line from the specified file.
- insln targetFileName line text : inserts a new line of text to the specified file.
- shwln targetFileName line : shows the contents of a specific line of text from the specified file.
- lnnum targetFileName : outputs the number of lines in the file specified.
- man : brings up the function guide.
- ext : exits the program.
```

This may or may not be detailed enough information for the user, will be tested on actual real world people, to see if they understand. Finally, we need to consider the case in which an unknown command is entered, where it will simply output that the command is not found. Now that is all determined the full code for taking the inputs can be constructed:

```
1 while(1)
2 {
3     printf(">>>");
4     char input[100];
5     fgets(input, 100, stdin);
6     input[strcspn(input, "\n")] = 0;
7
8     char delim[] = " ";
9
10    char *command = strtok(input, delim);
11
12    if (strcmp(command, "rnm")==0 || strcmp(command, "rename")==0){ /* rename: 2 parameters*/
13        char** parameters = getParam(input, 2, 0);
14        if (parameters != NULL){
15            renameFile(parameters[0], parameters[1]);
16        }
17    } else if (strcmp(command, "crt")==0 || strcmp(command, "create")==0){ /* create: 1 parameter */
18        char** parameters = getParam(input, 1, 0);
19        if (parameters != NULL){
20            createFile(parameters[0]);
21        }
22    } else if (strcmp(command, "cpy")==0 || strcmp(command, "copy")==0){ /* copy: 2 parameter */
```

```

23     char** parameters = getParam(input, 2, 0);
24     if (parameters != NULL){
25         copyFile(parameters[0], parameters[1]);
26     }
27 } else if (strcmp(command,"del")==0 || strcmp(command,"delete")==0){ /* delete: 1 parameter
28 */
29     char** parameters = getParam(input, 1, 0);
30     if (parameters != NULL){
31         deleteFile(parameters[0]);
32     }
33 } else if (strcmp(command,"shw")==0 || strcmp(command,"show")==0){ /* show: 1 parameter */
34     char** parameters = getParam(input, 1, 0);
35     if (parameters != NULL){
36         showFile(parameters[0]);
37     }
38 } else if (strcmp(command,"appln")==0 || strcmp(command,"appendLine")==0){ /* append line: 2
39 parameter */
40     char** parameters = getParam(input, 2, 1);
41     if (parameters != NULL){
42         appendLine(parameters[0], parameters[1]);
43     }
44 } else if (strcmp(command,"delln")==0 || strcmp(command,"deleteLine")==0){ /* delete line: 2
45 parameter */
46     char** parameters = getParam(input, 2, 0);
47     if (parameters != NULL){
48         if (intCheck(parameters[1]) == 1){
49             deleteLine(parameters[0], atoi(parameters[1]));
50         } else {
51             printf("Inappropriate input for line number.\n");
52         }
53     }
54 } else if (strcmp(command,"insln")==0 || strcmp(command,"insertLine")==0){ /* insert line: 3
55 parameter */
56     char** parameters = getParam(input, 3, 1);
57     if (parameters != NULL){
58         if (intCheck(parameters[1]) == 1){
59             insertLine(parameters[0], atoi(parameters[1]), parameters[2]);
60         } else {
61             printf("Inappropriate input for line number.\n");
62         }
63     }
64 } else if (strcmp(command,"shwln")==0 || strcmp(command,"showLine")==0){ /* show line: 2
65 parameter */
66     char** parameters = getParam(input, 2, 0);
67     if (parameters != NULL){
68         if (intCheck(parameters[1]) == 1){
69             showLine(parameters[0], atoi(parameters[1]));
70         } else {
71             printf("Inappropriate input for line number.\n");
72         }
73     }
74 } else if (strcmp(command,"lnnum")==0 || strcmp(command,"lineNum")==0){ /* number of lines:
75 1 parameter */
76     char** parameters = getParam(input, 1, 0);
77     if (parameters != NULL){
78         printf("The number of lines in the file is: %d\n", numLines(parameters[0]));
79     }
80 } else if (strcmp(command, "ext")==0 || strcmp(command,"exit")==0){
81     break;
82 } else if (strcmp(command, "man")==0){
83     printf("%s", introText);
84 } else {
85     printf("Command not found.\n");
86 }
87 }

```

This implementation, is far from perfect. If using other languages, it would be easily possible to pass in a function as a parameter to a function, and then a general form would be made to contain everything once the command has

been confirmed. Additionally, the code is lacking one feature, and that is in regards to the changelog.

2.1.4 Changelog design

The reason the changelog has been put off for so long is because this is the area that requires effectively every other section to be completed, since the changelog is storing the data of every operation taken. By referring to the FAQ we can see the absolute essentials of what we need to include in the changelog. We need to have the operation taken on each file, the name of the file it operated on, and the number of lines present in each file. Other than that there are any number of additional features and values we could add to the changelog. For the time being, there is only one other value worth adding, and that is the time that the operation was done. First we will want to start out by defining a log file at the start of the program, finding a way to ensure that it does not get rewritten each time the program is loaded. Following this we will need to implement a method to be called after each function in the main program, which will take in the name of the file it worked on, and the operation it worked on. Within the function then, it will be able to use the numLines function, as well as the inbuilt functions in time.h to gather the other pieces of data. What is important to consider is what other pieces of data we might want to include in the changelog, and this is best covered when discussing the additional features, as there are useful applications of the changelog, dependent on it having certain pieces of data.

```
1 int addChangelog(char* command, char* name)
2 {
3     FILE * c1 = declareFile("ChangeLog.log", "a");
4     if (c1 == NULL){
5         printf("The file failed to found.");
6         return 0;
7     }
8     char log[100];
9     time_t t;
10    time(&t);
11
12    char lines[10];
13    sprintf(lines, "%d", numLines(name));
14    char* time = ctime(&t);
15    time[strcspn(time, "\n")] = 0;
16    sprintf(log, "%s, %s, %s, %s", time, command, name, lines);
17
18    appendLine("ChangeLog.log", log);
19    return 0;
20 }
```

This snippet of code is relatively simple in what it achieves. Firstly (once it has declared a file and so on), it will define what time and date that the file operation happened by using lines 9,10 and 14; all of them contributing to getting the string that is structured like so: Day Month Date Time Year; for example: Mon Jan 24 02:26:19 2022 (Yes I am cutting it rather close). Once that is gathered, we also define the string of lines using sprintf, making use of the numLines function defined earlier. Finally, we can use sprintf to add of these details to a log string, which is structured like so:

Date and time, Command name, Textfile operated on, number of lines in the file.

For example:

Mon Jan 24 02:26:19 2022, appln, goodbye.txt, 3

Each value will be separated by a comma and a space, which works in all cases for this set of data, as no value will include a comma space. The reason this base set of data was chosen is because it applied to all files, where as details like line operated on, or text added, do not apply to all file operations. We use this structure and appendLine function to ensure that we add this line of detail to the changelog file. This way, when outputting it, it will give the sequence of operations in order, each line representing a particular file operation. This concise format allows for a fairly simple and understandable changelog, and with that addition command, 90% of the work is done for us. This command will then be put after every single function usage in main. In the large snippet in the previous user input section, the changelog code will be added in.

The final show changelog code then will be (currently) as easy as showFile, as we are just going through the file systematically and showing the user the contents. There is one small detail to be added though. We need the code to identify if this is the first time that the code is being run. The method to implement to do this is to have there

be a check to see if a changelog file exists. If a file does exist, it will assume that this is not the first time that it is being opened, hence it does not need the intro text. This can be done using the access function, which checks if the file exists.

2.1.5 Testing

2.1.5.1 Erroneous inputs

Now that all of the major pieces are in place, the only thing left to do is test the whole program, and try to ensure that there is no inputs or outputs that we do not want. The bulk of these are centered around the user input, as there are quite a few cases that we need to account for. The first area tested in the inputs was what the program did upon just pressing enter, or adding whitespaces and then pressing enter. These both return segmentation faults which is very much unwanted. The solution to this is an if statement that requires the input to have a length greater than 0 (as in it isn't just pressing enter), and another condition that ensures that the first character is not a whitespace. If it is, then that implies that the input is falsely constructed, and so does not return anything.

The next tests focused on how the program handled inappropriate inputs, and this was tested on a few categories. First, I went through each number of parameters per function; createFile having one parameter, copyFile having two parameters, and insertLine having three parameters. Here we can see that too many or too few parameters can be identified; though insertLine is unable to have too many parameters, since it classes everything after the final space as text.

```
>>>crt
Not enough parameters.
>>>crt newfile.txt hello
Too many parameters.
>>>cpy newfile.txt
Not enough parameters.
>>>cpy newfile.txt newnewfile.txt 5
Too many parameters.
>>>insln newfile.txt
Not enough parameters.
>>>insln newfile.txt 5
Not enough parameters.
>>>insln newfile.txt 5 This text here
```

Figure 8: Error testing on number of parameters.

The next area of testing is where an error came about. Here I was testing certain functions, and came to realise even when a text file did not exist in the directory, it would operate it out as the code for creating a file, is incredibly similar to reading from one. So, we make use of the access function once more. This is broken down into two parameters; the name of the text file to be checked, and the mode we want to check for. As we are checking for existence, we use F_OK. This will then go about in ensuring that if the file does not exist, it will not go about making it. However, we need to make an exception, and that is if the command is createFile. This is because the whole point of the file, is that it is trying to make a file, which does not exist yet. By putting this as an exception means we want to check whether the command is crt or create; or alternatively, we can check if the already checked function begins with cr, as they are the only two inputs in it currently that do it to any degree(crt and create). This designs itself as this code snippet:

```
1 if (input[0] != 'c' && input[1] != 'r' && input[2] != 't'){
2     if(access(arr[0], F_OK) != 0){
3         printf("File not found.\n");
4         return NULL;
5     }
6 }
```

```

>>>
>>>
>>>a
Command not found.
>>>345
Command not found.
>>>thj
Command not found.
>>>fd
Command not found.
>>>f
Command not found.
>>>apln notInDir.txt append line.
File not found.
>>>shw notInDir.txt
File not found.
>>>

```

Figure 9: Error testing command and file existence.

The screenshot shows this quite well, the first two lines showing what happens when you press enter without entering any characters/entering only spaces. Following this we see that anything entered that isn't a command just returns 'Command not found'. Then we can finally see that some functions that take named file arguments, that if the file is not in the directory, then it will not call the function. However, as we make an exception for create function, we need to put something else in place. This is because if we enter something that isn't a text file for the create parameter, it will create an undefined file.

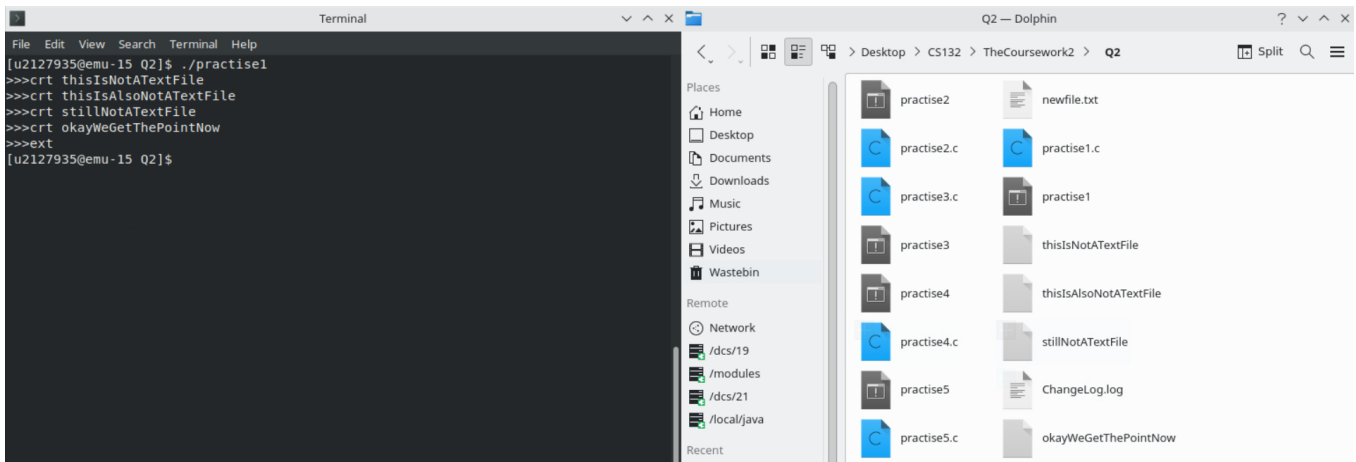


Figure 10: Error testing on createFile.

As we can see in the screenshot above, we find that the text files effectively cannot exist. The solution is we can check the last four characters of the string, and check if it is '.txt'. Then, if it isn't, we can use the excess space of the input to go and add '.txt'. The issue with doing this, is that then this changes the pointer tracking the number of parameters. It is incredibly weird, and so the simpler route will be taken, by just implementing an error message if .txt isn't included in the input.

```

1 if (input[0] == 'c' && input[1] == 'r'){
2     int len = strlen(arr[0]);
3     char sub[4];
4     memcpy(sub, &arr[0][len-4], 4);
5     sub[4] = 0;
6     if (strcmp(sub, ".txt") != 0){
7         printf("Enter a valid file name with .txt at the end.\n");
8     }

```

```

9  } else {
10     if(access(arr[0], F_OK) != 0){
11         printf("File not found.\n");
12         return NULL;
13     }
14 }

```

```

>>>crt okaywhatsup
Enter a valid file name with .txt at the end.
>>>crt okaywhatsip.txt
>>>

```

Figure 11: Error result from createFile.

2.1.5.2 Size and memory allocation

The final area of erroneous cases, is in surrounding the limitations of the hardware and software. This surrounds the limitations of numbers of characters, number of lines, etcetera. There are three sections to look at: memory allocation within C, max length of lines in text files, and max number of lines in text files. Now, I am very certain that there is a limit, as several times throughout this coding process, I have bloated a text file to the point it broke the computer upon opening. However, the actual specifications don't seem to be that clear. By using the following command

```
file -i fileToCheck.txt
```

We get the output

```
fileToCheck.txt: text/plain; charset=us-ascii
```

This means the character set used to encode the data is ASCII (as expected). By going on to a post we can find out some information about textfiles, and that there is a size limit... sort of. For lines, there is no explicit size limit, as if the number of characters exceeds the limit, it will just append to the next line. Lines however do have an integer limit of 2147483647 (or 2^{31}), at least on certain machines (also information may be outdated. At this point, when we are getting certain sizes, and we are dealing with extremes of this magnitude, we will make the approximation, that sizes of the files are unlimited. This wouldn't make sense to most, however, the reason we are doing this, as not only can we add only a line at a time, and it is difficult to gather this sort of information for ever machine and system, but it also is dependent on C. C is the middleman here, and the reason that is a problem is that it can only accept so many character per input. Currently, as you will have seen in the User input section, the length of the input is 100. However, that isn't necessarily enough, especially when we consider the text inputs, or files with absurdly long names. So to test the effects, I lowered the number of characters it takes as an input, to 20, to go and see what would occur.

For the sample input I went with:

```
>>>appln hello.txt 12345678901234567890123456789012345678901234567890
```

This input has 16 characters used to accept the command, and the filename; and then 50 characters for the text to be added (51 if including the space between them). What happens is quite fascinating. The characters 123 are appended to the text file 'hello.txt'. Following this, there are three outputs, all saying

```
>>>Command not found.
```

This is the error message given by my program when there is no command that matches the string entered. This means that when pressing enter on a string that exceeds the set number of characters, it means that it will execute the command as expected within the range set, and then proceed to try again in the next range, and again and again until it cannot execute any more. The maximum number of characters from the command entered is (currently) 10, which relative to how much can be entered for the line and the size of the text file... isn't very much. This means we can exclude it from future calculations. Next is concerning the length of a name for a text file. By a quick Google search it said that they it was 255 bytes, so less than 255 characters. By copying and pasting a string of 255 characters:

'Lorem ipsum dolor sit amet, nonummy ligula volutpat hac integer nonummy. Suspendisse ultricies, congue etiam tellus, erat libero, nulla eleifend, mauris pellentesque. Suspendisse integer praesent vel, integer gravida mauris, fringilla vehicula lacinia non'

I proceeded to fiddle with the size of this until it didn't return a text file, eventually finding the size to be: 245 characters. Then for the length of line, the size limit is effectively whatever we decide it to be. This is all important as if we look in the `getParam` function, we allocated a certain amount of memory (100 characters) for each parameter (not the command). However, since we know that there is no limit, we need to figure out how much memory we want to allocate. C itself has many factors to how much memory it can allocate, and so to base it off of that seems to be a poor choice; additionally it seems like a poor idea to allocated 100 characters to something like line number. This is where the difficulty in what decision to make comes. The way in which the `getParam` function was designed, was to make it apply to all functions, however, if we set line parameters (the second parameter) to be a certain limit - lets say 10 characters - then the `copyFile` function becomes incredibly limited. Perhaps we could implement a variable similar to text, which uses a simple Boolean logic of 0 being false, and 1 being true, defining what parameter is an integer input. This allows for the code to have greater modularity, since it is easier to implement other functions that use lines. However, this bloats the `getParam` method, filling it with if statements and cases. Now, there is another solution, and that is to reallocate memory to a smaller amount, effectively freeing up memory to be used else where. This can be done using the `realloc` function, specifying a previously defined section of memory, and a new size to give it. The only thing is, whenever `realloc` gets involved, so does segmentation faults. Due to a short span of time, and the limitations of my knowledge, the memory is going to be left to be inappropriately allocated, with a lot of wasted space. However a small improvement will be made, by specifying the size of the text later on to a much larger size of 250 characters, whilst keeping the command and line/copying parameter to a decent, yet smaller size of 100 characters.

2.1.5.3 Function edge cases

When testing execution, there were a few edge cases in regard to the line operations that reared their heads. Firstly for the `deleteLine` function, there were two cases to deal with. I initially had to ensure that if the delete function was enacted on a line past the EOF, then it wouldn't do anything. This is sorted by the program breaking after EOF. This needed to work for the line the EOF was one, and this was suitably done by adding an extra break in the function. The next area to consider was to ensure that it was deleting lines that had no content on them. For this a change needed to be made, as it would leave the file in the state it was in prior to the operation. The change that needed to be made was to check whether the first character was a newline character. If it was, then it would need to break without appending a newline.

The next function to deal with was the `insertLine`, which had to tackle similar areas. Starting with how it handles the end of a file, this is sorted by the fact that the appending is handled in two chunks, and if on the same line as the EOF a new line is added, which is where the data is appended to. The other case is with blank lines; however, this also handles itself, though maybe not in the ideal way. So this shifts up everything ahead of it, which can be the intent, unless we don't want to shift the blank line, and instead fill it in. However, I have come to the conclusion it is more useful to shift everything, and then delete that line + 1, than build a function more specific toward that functionality.

2.1.5.4 Execution

Now that the extreme cases are dealt with, this section will go through each of the functions, displaying how they work with an example, mostly tackling cases as expected: expressing the base functionality of the program. Starting with an empty directory, with only the C file, we first run the program, and it pulls up the newly designed with manual of all of the commands. After contact with a friend - a student doing mathematics - they said that the intro was descriptive, but didn't make it clear enough what commands were what, so by adding on that extra detail, as well as the extra commands I missed, we finally get this interface:

Commands take parameters separated by spaces; aside from text which can continue indefinitely.

The list of commands and parameters are below:

- rename: `rnm targetFileName newFileName` : renames the specified file, with the newly specified name.
- create: `crt fileName` : creates a text file with the name specified.
- copy: `cpy targetFileName newFileName` : copies the specified file, giving it the newly specified name.
- delete: `del targetFileName` : deletes the file with the name specified.
- show: `shw targetFileName` : shows the contents of the file with the name specified.
- appendLine: `appln targetFileName text` : appends a new line of text to the specified file.
- deleteLine: `delln targetFileName line` : deletes a specified line from the specified file.
- insertLine: `insln targetFileName line text` : inserts a new line of text to the specified file.
- showLine: `shwln targetFileName line` : shows the contents of a specific line of text from the specified file.
- lineNum: `lnnum targetFileName` : outputs the number of lines in the file specified.
- showChangelog: `shwCl` : shows the contents of the changelog file.
- manual: `man` : brings up the function guide.
- exit: `ext` : exits the program.

There I proceed to call three commands:

```
>>>crt newFile.txt
```

```
>>>appln newFile.txt These are the reasons that python is the best programming language:
```

```
>>>insln newFile.txt 2 - The language can be used by new and experienced users alike
```

The first command creates the text file as per standard. The appendLine function then adds to the first line, as its pointer is set at the end of the file. The next command creates a new file, copies the current contents across, and then inserts the line where it is supposed to go, deleting the old file to 'hide the evidence'. The text file produced gives this text:

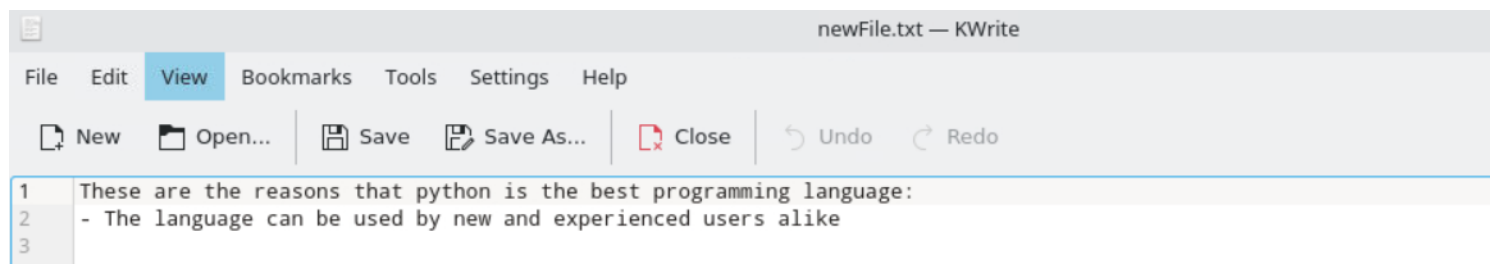


Figure 12: Execution of crt, appln and insln.

We next proceed to call:

```
>>>cpy newFile.txt copyFile.txt
```

```
>>>del newFile.txt
```

```
>>>shwln copyFile.txt 2
```

The copy command acts as intended, creating a new file with the specified name, and then copying over every character. Then using the inbuilt delete command the original file is removed. Finally we ask for the output on line 2, and it returns:

```
- The language can be used by new and experienced users alike
```

As expected. Finally we call these commands, gathering these outputs:

```
>>>lnum copyFile.txt
The number of lines in the file is: 2
>>>delln copyFile.txt 2
>>>shw copyFile.txt
These are the reasons that python is the best programming language:
>>>del copyFile.txt
>>>shwCl
Wed Jan 26 21:59:32 2022, appln, newFile.txt, 1
Wed Jan 26 22:00:32 2022, insln, newFile.txt, 2
Wed Jan 26 22:08:09 2022, cpy, newFile.txt, 2
Wed Jan 26 22:08:35 2022, del, newFile.txt, 0
Wed Jan 26 22:09:17 2022, shwln, copyFile.txt, 2
Wed Jan 26 22:13:04 2022, lnum, copyFile.txt, 2
Wed Jan 26 22:13:25 2022, delln, copyFile.txt, 2
Wed Jan 26 22:13:53 2022, shw, copyFile.txt, 2
Wed Jan 26 22:14:04 2022, del, copyFile.txt, 0
>>>
```

These all act as we want them too, with the final state of the directory only having the program and the changelog, which holds everything what we have performed up until this point. It was in these execution paths that most errors were found, and how they were dealt with makes for great content; but with less than 12 hours to go until it is due in, something take higher priority.

2.1.6 Conclusion

To conclude, much of this project was successful. The functions largely operate entirely as you would expect, and there is very little (that I have encountered) that warrants more changing to a user who would briefly use this program. Despite that achievement though, there are many areas that feel lacking in optimisation. This is mostly in memory organisation and use of space, it seeming at times as if I were treating the machine to have an unlimited amount.

Areas in which the program could use improvement is more than anything the allocation of memory. Starting by using a large number of characters, with potential for very few to be used, my lack of understanding of C memory management comes on full display. As discussed in the Size and memory allocation segment, there was an effort in looking into harnessing the full prowess of C to negate the amount of wasted space, and free more memory without data leaks, but it didn't get too far as I described: whenever malloc gets involved, so does segmentation faults. However, my memory issues aren't limited to just taking inputs, it also falters in the algorithms used. I learnt little too late about the methods that allow for the manipulation of characters and pointers within text files, and because of this it meant that a very poor method was used to implement the functions of insertLine and deleteLine. Poor as in the memory space, as the space taken up by creating a file, copying the contents, and then deleting a previous file can be taxing for large text files. There are other limitations present as well, such a making it so you are unable to create text files with spaces in them, which is extremely convenient for my program, but is the removal of an existing feature.

What is lacking in memory optimisation though, is hopefully made up for by series of areas that were genuinely successful. The first thing to acknowledge is that all functions worked, even on extreme cases where I was trying to break them. Whilst not every single case where I tried to break the function is detailed here due to time, the efforts made availed code that is quite strong and difficult (not impossible) to break. Furthermore, every function runs in at least $O(n)$ time, as there is no point in which multiple loops have to occur inside of a function; the fact many of the more complex functions abuse memory, allow for them to have much more concise and elegant solutions algorithmically, as pointers no longer hinder the methods. Most of all though, I am incredibly happy in the user interface and how it interacts with the user. The use of the parameters opened the door up for just far easier command execution that flowed much more seamlessly than it would otherwise, and opens up the doorway for more functions in the next task.

Hence, my final thoughts on this first part of the coursework is that it was a great introduction into file manipulation,

and how frustrating pointers and memory allocation can be. Whilst there are many areas for improvement - especially at larger sizes - the algorithms designed work for the scope we are set at.

2.2 b.

Implement two additional operations or extensions to the operations listed above. You should provide justifications for the usefulness of the functionalities you introduce.

2.2.1 Decomposition

This section of the project is where it really opens out, a lot. As opposed to the first part, there is nothing to decompose, aside from the functions that I decide to do. When thinking about what things a text editor could use, I was the whole time also thinking about what would be interesting. Not just a function that would be the same old string manipulation I had been doing this whole time, but something to make me think, something that I could design and have fun designing. Considering all of these points, I found a selection of possible functions I could implement.

- Find: Enters a text file and attempts to locate a specified string, returning word position, and line it is on.
 - + Useful function to the user, letting them find different instances of text.
 - + Has room to include more detail, and for it to be expanded on.
 - Requires a lot of string manipulation.
 - Relatively uninteresting to implement.
- Evaluate Line: This function would locate a line in a file, and if it is a mathematical expression, then it will evaluate it to the correct expression.
 - + Very interesting idea, requiring more than just string manipulation.
 - + Has room to be expanded upon, adding additional functions and functionality to it.
 - + Can be useful if the user has equations wanting to be solved.
 - String manipulation to error check.
 - Very specific usage.
- Encryption: Encodes the data of a text file with an option of ciphers and encryption methods.
 - + Useful to the user, as it allows for data to be effectively protected, as if it had a password.
 - + Has a lot of room for expansion, using various ciphers: Vigenère, Caesar, Binary/Morse/OtherBases, even some own custom made.
 - Requires a lot of string manipulation, and a good understanding of each cipher.
 - User could forget passcode, and then we would really be screwed.
 - Would only be single key authentication, two factor is too difficult for C.
- Undo/Redo: It undos and redos... I really do not know how else to describe it.
 - + Most useful feature, how many times have we accidentally deleted a file that we didn't want too?
 - + Is complex and interesting to design.
 - Is complex and difficult to design.
 - Requires a detailed changelog.
- Directory commands: This implements the commands to create, move to and delete directories as well as text files.
 - + Useful feature for creating a more interactive command line.
 - Requires a strong understanding of linux, bash and how all the commands work.
 - Pushes away from the focus of a text editor.
- Noughts and Crosses: Creates a text file with a board; then either two players or a single player against a bot can play noughts and crosses by specifying position in command line (could also work for battle-ships).
 - + Useful for if the user or Matt gets bored whilst marking.
 - + Level of complexity from updating the file and the difficulty of the bot.
 - Really is not that useful to this being a text editor.
- Command line functions: commands like clear (refresh command line), list (list off text files in directory), etc.
 - + Incredibly useful for me, this series of utility commands would have made my whole life so much easier.
 - + Likely would also make the user's life greater as well.
 - Could be incredibly simple dependent on inbuilt commands.

The one that above all else, was the evaluate line function, but that one is more interesting to talk about, so we will save the best until last. The *other* function that caught my interest was the undo and redo. This was because

it was undoubtedly the most useful function here. Not only for me, but for other users, an undo and redo function is something that would entirely change the way the program works fundamentally, but also how you could operate and interact with it. There are two ways that I could see this working; one is to have some external storage system outside of changelog, which stores the previous state of the directory. This method may be limited to a single undo, unless there is a way around it I cannot see... however, that route is incredibly boring. That method requires lots of research into specific functions of C, and how to store data... its not what I was looking for. The other way though, is far more interesting.

Imagine each operation as a function, a mathematical function with a domain and range. Every operation or 'function' performed theoretically then has a function or a series of functions that would do the inverse. So then I started thinking of mapping each operation, to a series of inverse operations, to replicate the original state, based off of what is contained in the changelog.

Function	Inverse Function
renameFile(name, newname)	rename(newname, name)
createFile(name)	deleteFile(name)
copyFile(copyping, newname)	deleteFile(newname)
showFile(name)	NULL
appendLine(name, text)	deleteLine(name, numLines(name)-1)
insertLine(name, line, text)	deleteLine(line)
deleteLine(name, line)	insertLine(name, line, text)
showLine(name, line)	NULL
numLines(name)	NULL
showChangelog()	NULL

As we can see from above, most functions are quite simple. Obviously, functions that do not change the state of the directory have no inverse (unless I can create time travel and prevent the function from ever being called). Then we also have very simple one to one functions; where createFile can simply be counteracted by deleteFile, and so on and so forth. Next we can look at the more restrictive and difficult functions, such as deleteLine, which requires us to have stored the line operated on, and the text held on that line. However, other than a few changes, none of these seem so difficult to implement... Until you realise I have neglected to include one. The function deleteFile - the one that inspired this in the first place - is quite a bit more difficult. Not in the basic operations performed, rather in the storing of data. The advantage to housing a series of states is that all file content can be held safely, however here, by limiting ourselves to the changelog, or even other text files, it can become incredibly uneconomical to store entire text files. This problem doubles when we have to consider redo functions, as then we have no basis of time for how long we must hold the data. Once again, this is no big deal for smaller text files, but for once massive in size, it would become a nightmare to hold and store, even just for a single session. This problem, combined with the lack of time that I have left to implement it, is why this will not be chosen as one of the functions. However, I still wanted to write down my reasoning and potential design, as this was the original endgame of the project.

2.2.2 List files

Since I am low on time, I am going to engage in a slightly less engaging function, and that is a function to list the text files in the current directory. I choose this one as, it is really useful to have; very much in the vein of ls in the linux terminal, it lets the user have a menu to see what text files they are interacting with. Before this, users have had to repeatedly look back and forth at what files existed, but by implementing this function, it allows for the actual selection of files. Hence, beginning this I had to learn how to access all files in the directory; following this I stumbled across a snippet of code making this far easier. Using the library dirent.h we can define a struct which contains details about the directories in the current directories. Using certain commands from it we can get all names of files in the current directory; as can be see in the code snippet:

```

1 #include <dirent.h>
2 #include <stdio.h>
3
4 int list()
5 {
6     DIR *d;
7     struct dirent *dir;
8     d = opendir(".");
9     if (d)
10    {

```

```

11     while ((dir = readdir(d)) != NULL)
12     {
13         printf("%s\n", dir->d_name);
14     }
15     closedir(d);
16 }
17 return(0);
18 }

```

With this, we can combine it with the program from earlier that checked whether the name entered in createFile function has .txt at the end. By combining them we can have it only output text files, which is what we want. However, I thought that wasn't useful enough, and so in addition to the names of files, it will also make use of the numLines function we made, to output the number of lines; which helps the user know the scope of the file.

```

1 int list()
2 {
3
4     DIR *d;
5     struct dirent *dir;
6     d = opendir(".");
7     if (d)
8     {
9         while ((dir = readdir(d)) != NULL)
10        {
11            char* name = dir->d_name;
12            int len = strlen(name);
13            char sub[4];
14            memcpy(sub, &name[len-4], 4);
15            sub[4] = 0;
16            if (strcmp(sub, ".txt") == 0){
17                int lines = numLines(name);
18                printf("%s, %d lines\n", name, lines);
19            }
20        }
21        closedir(d);
22    }
23    return(0);
24 }

```

2.2.3 Evaluate line

2.2.3.1 Decomposition

Whilst I was unable to do the more interesting undo and redo, there was one function I was not missing out on. Everyday we use calculators, programming languages, interpreters etcetera; but how do they interpret? Through the use of parsing. Parsing is syntax analysis and to a lesser extent that is what we have been doing by taking the inputs. However, I have always wanted to know how to gather up mathematical terms and spit out something meaningful out of them. What was the method to the madness? Due to the limited time once more, I set out to implement a maths parser that will find numerical solutions to problems involving addition, subtraction, multiplication, and division. Though with a problem so large, where do I even begin to start tackling it. As with all computer science questions, the answer lies in Google. Going to the classic GeeksforGeeks website, expression evaluation is an algorithm depicted, yet not in any form that C will be able to handle. In this case, I need to break down the algorithm, step by step, and finding out how to truly implement this program.

The GeeksforGeeks webpage does have a list of what needs to be done, where everything is quite abstracted away. From that series of instructions, I translated it not into C terminology, but first into more C like syntax. This was done so that I had a frame of reference for what to actually use.

```

1 int priority(operator)
2 {
3     if '+' or '-'{
4         return 1
5     } else if '*' or '/' {
6         return 2
7     } else {
8         return 0
9     }

```

```

10 }
11
12 int apply(x, y, operator)
13 {
14     if '+' {return x+y}
15     else if '-' {return x-y}
16     else if '*' {return x*y}
17     else if '/' {return x/y}
18 }
19
20 int update(values, ops)
21 {
22     y = values.pop()
23     x = values.pop()
24     op = ops.pop()
25     values.push(apply(x, y, op))
26 }
27
28 int evaluate(exp) //exp for expression
29 {
30     values = stack()
31     ops = stack() //ops for operators
32     i = 0
33     while (i < strlen(exp))
34     {
35         if exp[i] == whitespace{i = i + 1}
36         else if (exp[i] == '('){
37             ops.push(exp[i])
38         } else if (isdigit(exp[i]) == 1){
39             val = 0
40             while (i < strlen(exp) and isdigit(exp[i]) == 1)
41             {
42                 val = val * 10 + int(exp[i])
43                 i = i + 1
44             }
45             values.push(val)
46             i = i - 1
47         } else if (exp[i] == ')'){
48             while (size(ops) != 0 and ops[top] != '(')
49             {
50                 update(values, ops)
51             }
52         } else if (exp[i] == '+' or '-' or '*' or '/') {
53             while (size(ops) != 0 and ops[top] is greatest priority)
54             {
55                 update(values, ops)
56             }
57             ops.push(exp[i])
58         }
59         i = i + 1;
60     }
61     while (strlen(ops) != 0)
62     {
63         update(values, ops)
64     }
65
66     return values[top]
67 }
68 }

```

Now just looking at that block of text is intimidating beyond belief, but working through it, it begins to make a lot more sense. The algorithm is the Shunting Yard algorithm, which effectively aims to make expressions that are in infix notation, the way that we write expressions: $1 + 2$, $4 * 5$, $2 - 3$ etc. and makes them postfix: $1\ 2\ +$, $4\ 5\ *$, $2\ 3\ -$ etc.

1. Firstly we define the functions of priority, apply and update.
Priority is there to represent order of operations (think to bidmas).
Apply is there to carry out the operation specified.
Update is there to update the values stack of an operation.

2. Next we begin to traverse through the expression given, indicated by i.
 - If the character is a white space, we ignore and move on.
 - If the character is an opening bracket '(', we push it to the operation stack.
 - If the character is a digit
 - (a) Traverse the list until it has the full base 10 numerical value.
 - (b) Push the value on to the value stack.
 - If the character is a closing bracket ')'
 - (a) Performs all operations in the brackets, using update to push to value stack.
 - (b) Pop values stack.
 - If the character is an operator
 - (a) Performs all operations of greatest priority, using update to push to value stack.
 - (b) Pushes the operation on to the value stack.
3. Traverse to next character.
4. Once fully traversed, updates the value stack until there are no more operations.
5. Return the top of the value stack.

This algorithm broken down we can now see how it works. It continually adds to the stack, gathering the whole part of an expression, before clamping down and evaluating each part. In some ways it divides and conquers, making the problem smaller and smaller until the answer is arrived at.

2.2.3.2 Worked examples

To demonstrate the algorithm, the best way to show it is with some examples, starting off with something simple:

```

1 exp = "5+8"
2 exp[0] = 5
3   exp[0] is digit
4   val = 5
5   values.push(5)
6 exp[1] = +
7   exp[1] is operator
8   ops.push(+)
9 exp[2] = 8
10  exp[2] is digit
11  ops.push(8)
12
13 values = [5, 8]
14 ops = [+]
15 update(values, ops)
16 values = [13]
17 ops = []
18
19 return 13

```

Here, this shows the basic steps that the algorithm takes, yet we can't see how it deals with priority, and order.

```

1 exp = "2+3*64/(4+4)"
2 exp[0] = 2
3   is digit
4   values.push(2)
5 exp[1] = +
6   is operator
7   ops.push(+)
8 exp[2] = 3
9   values.push(3)
10 exp[3] = *
11   ops.push(*)
12 exp[4] = 6
13   values.push(64)
14 exp[6] = /
15   priority(/) = priority(ops[top])
16   values = [2, 3, 64]
17   ops = [+, *]
18   update(values, ops)
19   values = [2, 192]
20   ops = [+]

```

```

21     ops.push(/)
22 exp[7] = (
23     ops.push()
24 exp[8] = 4
25     values.push(4)
26 exp[9] = +
27     ops.push(+)
28 exp[10] = 4
29     values.push(4)
30 exp[11] = )
31     ops[top] != (
32         values = [2, 192, 4, 4]
33         ops = [+, /, (, +]
34         update(values, ops)
35         values = [2, 192, 8]
36         ops = [+, /, (]
37     ops.pop()
38
39 values = [2, 192, 8]
40 ops = [+, /]
41 update(values, ops)
42 values = [2, 24]
43 ops = [+]
44 update(values, ops)
45 values = [26]
46 ops = []
47
48 return values[top]

```

These examples help demonstrate how the program works beyond words, and it truly starts to make sense; though implementing it into the most awkward language concerning strings, may be difficult.

2.2.3.3 Design

The first two functions are easy enough to implement in C, since we know that it is a simple function identifying a character, and returning a value from it.

```

1 int priority(char operator)
2 {
3     if (operator == '+' || operator == '-') {
4         return 1;
5     } else if (operator == '*' || operator == '/') {
6         return 2;
7     } else {
8         return 0;
9     }
10 }
11
12 int apply(int x, int y, char operator)
13 {
14     if (operator == '+') {
15         return (x+y);
16     } else if (operator == '-') {
17         return (x-y);
18     } else if (operator == '*') {
19         return (x*y);
20     } else if (operator == '/') {
21         return (x/y);
22     }
23 }

```

With those out the way though, we are out of the frying pan, and into the fire. The fire in this case, being that C doesn't have stacks. My initial plan was to implement two functions that can operate on arrays, a push and a pop function. Pushing just increments the top, and sets the value specified to it; whereas popping returns the value we want, and decrements the top (effectively removing it from the array). However, if we wanted to do this, we would need to mess with interesting addressing systems, and also implement two functions. C can't handle multiple types, so we would then need to implement a push and pop for integers (value stack) and characters (operator stack).

That is why I am fortunate there is a way around this. Defining two stacks at the start of the function, and setting the top to equal -1 (indicating it is empty), then whenever we want to push a value on to the stack we can do:

```
1 stack[++top] = value;
```

And similarly for pushing:

```
1 value = stack[top--];
```

This makes use of the fact that operations before an integer act before anything else (maximum priority), and anything after, occurs after all operations. This leaves the push and pop of stacks not only data type neutral, but also limited to a single line. However, in that victory we do find a limit that implementing the update function leads to a lot of icky pointer addressing problems; leaving it best to have all four lines implemented in one- which isn't the biggest sacrifice fortunately.

Upon dealing with the stack problem, it seemingly becomes trivial. Now we can add and subtract to stacks, and since it is all so abstracted we only deal with characters and numbers. Although I could have just left it here, posted the code, and got a full 8 hours sleep. Instead, I knew this needed expanding. The algorithm worked, no issues in any inputs; aside from the fact it was all integers. The algorithm rounded/truncated whenever a decimal value was found; moreover there was no method to deal with decimal values; like what if I wanted to add 1.1 and 1.2? The way we could solve this was by changing all integer type definitions to double type definitions. Once we had done this, we needed to amend the method that values were added to the stack. Currently, it parses through each number until reaching a non-digit, each time multiplying the current value by 10, and adding the new value.

```
1 exp = "123"
2 val = 0
3 exp[0] = 1
4     val = 0 + 1 = 1
5 exp[1] = 2
6     val = 1*10 + 2 = 12
7 exp[2] = 3
8     val = 12*10 + 3 = 123
```

This can be expanded, by effectively doing the opposite, once we pass a decimal point. Instead of multiplying the current val by 10, we divide the added character by 10^n , which we can define using the power function made previously.

Now we have expanded the domain to the positive rationals, it is now time to go backwards and add functionality for negative numbers. By making the values a double, it means that it can be negative inherently; however, the parser won't know what to do when encountering $5 * -5$, as it will add the negative to the operator stack. The solution to this I found was that assuming that the syntax was all correct, then we can conclude at no point will there be two operations next to each other, unless the following number is a negative. So, in the operator section, we make a check to see if the next character is -, if it is then we set a negative flag to equal 1, and increment an additional time, to skip that character. By doing this, we know to multiple the next value by negative 1, giving us the correct negative value. This all comes together to give the program:

```
1 double evaluate(char* expression)
2 {
3     double* values = malloc(100 * sizeof(double));
4     int valTop = -1;
5
6     char* ops = malloc(100 * sizeof(char));
7     int opsTop = -1;
8
9     int i = 0;
10    int neg = 0;
11    int size = strlen(expression);
12
13    while (i < size)
14    {
15        if (expression[i] == '('){
16            ops[++opsTop] = expression[i];
17            if (expression[i+1] == '-'){
18                neg = 1;
19                i = i + 1;
20            }
21        }
22    }
23 }
```

```

21 } else if (isdigit(expression[i])){
22     double val = 0;
23     int decimal = 0;
24     while (i < size && (isdigit(expression[i]) || expression[i] == '.'))
25     {
26         if (expression[i] == '.'){
27             decimal = decimal+1;
28         } else {
29             char digit[2];
30             digit[1] = '\0';
31             digit[0] = expression[i];
32
33             if (decimal == 0){
34                 val = val * 10 + atof(digit);
35             } else {
36                 double divisor = power(10, decimal);
37                 val = val + (atof(digit)/divisor);
38                 decimal = decimal + 1;
39             }
40         }
41         i = i + 1;
42     }
43
44     if (neg == 1){
45         val = -1*val;
46         neg = 0;
47     }
48     values[++valTop] = val;
49     i = i - 1;
50 } else if (expression[i] == ')'){
51     while (opsTop != -1 && ops[opsTop] != '(')
52     {
53         double y = values[valTop--];
54         double x = values[valTop--];
55         char op = ops[opsTop--];
56         values[++valTop] = apply(x, y, op);
57     }
58     opsTop--;
59 } else if (expression[i] == '+' || expression[i] == '-' || expression[i] == '*' || expression[
i] == '/') {
60     while (opsTop != -1 && priority(ops[opsTop]) >= priority(expression[i]))
61     {
62         double y = values[valTop--];
63         double x = values[valTop--];
64         char op = ops[opsTop--];
65         values[++valTop] = apply(x, y, op);
66     }
67     ops[++opsTop] = expression[i];
68
69     if (expression[i+1] == '-'){
70         neg = 1;
71         i = i + 1;
72     }
73 }
74 i = i + 1;
75 }
76
77 while (opsTop != -1)
78 {
79     double y = values[valTop--];
80     double x = values[valTop--];
81     char op = ops[opsTop--];
82     values[++valTop] = apply(x, y, op);
83 }
84 double final = values[valTop--];
85 return final;
86 }

```

However, this is still not the complete function. Whilst yes this gives an evaluation of the expression, but what we actually want is an evaluation of a specific line in a text file. To do this, we refer back to our file manipulation

tactics from part a.

```
1 double evaluateLine(char* name, int line)
2 {
3     int l = 1;
4     int i = 0;
5     char c;
6     char valueString[20];
7
8     FILE * fp = declareFile(name, "r");
9     if (fp == NULL){
10         printf("File not found.\n");
11         return 0;
12     }
13
14     char expression[100];
15
16     while(1)
17     {
18         c = fgetc(fp);
19
20         if (l == line){
21             expression[i] = c;
22             i = i + 1;
23         }
24
25         if (c == 10){
26             l = l + 1;
27         } else if (c == EOF){
28             break;
29         }
30     }
31
32     expression[strcspn(expression, "\n")] = 0;
33     double value = evaluate(expression);
34
35     if (value - (int)value == 0){
36         sprintf(valueString, "%d", (int)value);
37         printf("Line %d evaluates to %s\n", line, valueString);
38     } else {
39         sprintf(valueString, "%f", value);
40         printf("Line %d evaluates to %s\n", line, valueString);
41     }
42
43     deleteLine(name, line);
44     insertLine(name, line, valueString);
45     deleteLine(name, line+1);
46
47     fclose(fp);
48
49     return value;
50 }
```

2.2.4 Testing

When testing, the first function of List does not take any arguments, meaning there is very little to actually test. The only thing tested was that the number of lines outputted is the amount that is intended; which it is. Therefore, we can conclude that the list function is entirely successful. What isn't so easy, is ensuring that the evaluateLine function doesn't cause 50 segmentation faults. Now this is 5 hours before the due date, and so the testing is limited, however, the way to prevent any code breaking errors, is to scan through the expression gathered, and look for any syntax errors. There are a large number of areas we could focus upon, but now we want to clamp down on the large cases of breakage. There are three areas that may break the code:

1. Firstly any values that aren't recognised by the parser: all non-digits and all characters other than the operators and brackets.
2. All brackets opened are being closed.
3. Repetitions of operators in consecutive elements.

We can check these by fairly simply if statements, which uses a series of if statements to check each character against these parameters. However, there will be a lot of exceptions when tested harshly enough; but these conditions should remove most possible errors surrounding the inputs.

```

1 int validateExpression(char* expression)
2 {
3     if (isdigit(expression[0]) == 0 && expression[0] != '('){
4         return 0;
5     }
6
7     if (isdigit(expression[strlen(expression)-1]) == 0 && expression[strlen(expression)-1] != ')'){
8         return 0;
9     }
10
11     int i;
12     int opflag = 0;
13     int b = 0;
14
15
16     for (i=0; i < strlen(expression); i++)
17     {
18         if (isdigit(expression[i]) == 0 && expression[i] != '(' && expression[i] != ')' && expression[i]
19             != '.' && expression[i] != '+' && expression[i] != '-' && expression[i] != '/' && expression
20             [i] != '*'){
21             return 0;
22         }
23
24         if (expression[i] == '+' || expression[i] == '-' || expression[i] == '/' || expression[i] == '*
25             '){
26             if (expression[i+1] == '+' || expression[i+1] == '/' || expression[i+1] == '*'){
27                 return 0;
28             }
29         }
30
31         if (expression[i] == '('){
32             b = b + 1;
33         }
34
35         if (expression[i] == ')'){
36             b = b - 1;
37         }
38     }
39
40     if (b != 0){
41         return 0;
42     }
43     else {
44         return 1;
45     }
46 }

```

There are some known exceptions to the rule; these mostly involve exceeding the max value accepted for a double, although this is minimized by the double having a much greater maximum value. Additionally, placing decimal points in consecutive locations will also lead to errors, and unfortunately is something I did not get around to solving. The final area of insecurity in the parser is dividing by 0, which now I am writing I feel is almost sinful. Since there is no exception currently for when division by 0 occurs, there is a big risk of errors there. This cannot be fixed by banning a certain string either, since division by 0 could still be achieved through a different expression. 5/0 could be rejected, but 5/(2-2) would not be. The solution here is that when the apply function performs its calculation, it checks that its second parameter is not equal to 0 when executing division, causing it to all return 0. This will need work and tinkering to ensure it is properly implemented.

Now, during the whole process of creation, I was testing inputs and different strings for it to evaluate, but the big final test was given by a fellow computer scientist:

```
1+5-2*3+3*(2-12*60-92)*3*5+6*(3+3*(2-12*60-92)-1)
```

By entering this on to Google, we get the value -51018. Now if we plug this into my parser; we get the final value of:

```
>>>evaln newText.txt 1
Line 1 evaluates to -51018
The expression evaluates to: -51018.000000
>>>
```

Figure 13: Using the evaln function.

Which is exactly what we want! This victory is enough to convince me of (when given the actual syntax), the maths is parsed correctly.

2.2.5 Conclusion

In conclusion, this project has been a roller coaster of learning, stress, intrigue and poor time management. There were parts that could have been done far better with that little extra push earlier on, and yet regardless of all of the flaws, it is a project I am proud of. Whilst there were limitations; with poor use of memory throughout, combined with certain edge cases and syntax of the evaluateLine function not being implemented to its fullest. Despite all of that, there was a lot of success to be found. In regards to the last two functions, the first list function covers the functionality and usability that we desired, being incredibly intuitive and almost akin to the ls function in Linux. The evaluateLine function was arguably less relevant to a text editor, but in the specific cases it is useful, and the sheer cool factor surrounding the parser makes me think it was successful. A lot was learnt about C, and how memory allocation actually works. The decomposition and design aspects were done most effectively, whereas the testing could have been done better with an emphasis on testing extreme cases. Most of all though, I learnt not to cram coursework on the night before it is due.

3 Bibliography

Anon (2020a). C - Scope Rules - Tutorialspoint. [online] Tutorialspoint.com. Available at: https://www.tutorialspoint.com/cprogramming/c_scope_rules.htm

Anon (n.d.). C library function - strcat(). [online] www.tutorialspoint.com. Available at: https://www.tutorialspoint.com/c_standard_library/c_function_strcat.htm

Anon (2020b). C library function - strtok() - Tutorialspoint. [online] Tutorialspoint.com. Available at: https://www.tutorialspoint.com/c_standard_library/c_function_strtok.htm

Anon (n.d.). strncpy - C++ Reference. [online] www.cplusplus.com. Available at: <http://www.cplusplus.com/reference/cstring/strncpy/>

Bibi, K. (2021). Getline function in C. [online] linuxhint. Available at: <https://linuxhint.com/getline-function-c/>

Brain, M. and Pollette, C. (2000). The Basics of C Programming. [online] HowStuffWorks. Available at: <https://computer.howstuffworks.com/c17.htm>

dgookin (2015). The getline() Function — C For Dummies Blog. [online] C for Dummies Blog. Available at: <https://c-for-dummies.com/blog/?p=1112>

Distinct Dog (2020). check if string is number c Code Example. [online] www.codegrepper.com. Available at: <https://www.codegrepper.com/code-examples/c/check-if-string-is-number-c>

Donotalo (n.d.). Coding Games and Programming Challenges to Code Better. [online] CodinGame. Available at: <https://www.codingame.com/playgrounds/14213/how-to-play-with-strings-in-c/string-split#:~:text=Split%20a%20string%20into%20tokens%20%2D%20strtok&text=char%20str%5B%5D%20%3D%20%22strtok%20needs>

Edpresso Team (n.d.). How to convert a string to an integer in C. [online] Educative: Interactive Courses for Software Developers. Available at: <https://www.educative.io/edpresso/how-to-convert-a-string-to-an-integer-in-c>

Edpresso Team (2020). How to use the fgets() function in C. [online] Educative: Interactive Courses for Software Developers. Available at: <https://www.educative.io/edpresso/how-to-use-the-fgets-function-in-c>

Egan, D. (2019). Passing Strings in C — Dev Notes. [online] dev-notes.eu. Available at: <https://dev-notes.eu/2019/07/Passing-Strings-in-C/>

Fawcett, A. (2020). How to concatenate strings in C: a five minute guide. [online] Educative: Interactive Courses for Software Developers. Available at: <https://www.educative.io/blog/concatenate-string-c>

forgetcode.com. (n.d.). Copy one file to another file in C - Forget Code. [online] Available at: <https://forgetcode.com/c/577-copy-one-file-to-another-file>

fresh2refresh.com. (n.d.). C itoa () function — C Type Casting functions. [online] Available at: <https://fresh2refresh.com/c-programming/c-type-casting/c-itoa-function/>

GeeksforGeeks (2009). Power Set. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/power-set/>

GeeksforGeeks (2013). Expression Evaluation. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/expression-evaluation/>

GeeksforGeeks (2015). sprintf() in C. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/sprintf-in-c/>

GeeksforGeeks (2016). strtok() and strtok_r() functions in C with examples - GeeksforGeeks. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/strtok-strtok_r-functions-c-examples/

GeeksforGeeks (2017). fgetc() and fputc() in C. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/fgetc-fputc-c/>

GeeksforGeeks (2018a). C Program to find size of a File. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/c-program-find-size-file>

GeeksforGeeks (2018b). Recursive program to generate power set. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/recursive-program-to-generate-power-set/>

GeeksforGeeks (2019). Linear Search - GeeksforGeeks. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/linear-search/>

groups.google.com. (n.d.). Do ASCII text files have a record length limitation? [online] Available at: <https://groups.google.com/g/comp.editors/c/i3RpsSNiaqI>

Jon (n.d.). arrays - Counting the number of times a character occurs in a string in C. [online] Stack Overflow. Available at: <https://stackoverflow.com/questions/7349053/counting-the-number-of-times-a-character-occurs-in-a-string>

Juan, J. (2020). write string to file in c Code Example. [online] www.codegrepper.com. Available at: <https://www.codegrepper.com/code-examples/c/write+string+to+file+in+c>

Luk, T. (2020). Power Set Algorithm with Recursion or Bits. [online] Medium. Available at: <https://blog.devgenius.io/power-set-algorithm-with-recursion-or-bits-cc3ffcf0daa>

Pankaj (2018). C program to create a file and write data into file. [online] Codeforwin. Available at: <https://codeforwin.org/2018/01/c-program-create-file-write-contents.html>

Paul (2019). C Programming - read a file line by line with fgets and getline, implement a portable getline version — Solarian Programmer. [online] solarianprogrammer.com. Available at: <https://solarianprogrammer.com/2019/04/03/c-programming-read-file-lines-fgets-getline-implement-portable-getline/>

Programiz (2020). C Program to Read the First Line From a File. [online] www.programiz.com. Available at: <https://www.programiz.com/c-programming/examples/read-file>

Programiz (n.d.). C strcat() - C Standard Library. [online] www.programiz.com. Available at: <https://www.programiz.com/c-programming/library-function/string.h/strcat>

RIP (2019). C Language Tutorial => Get lines from a file using getline(). [online] riptutorial.com. Available at: <https://riptutorial.com/c/example/8274/get-lines-from-a-file-using-getline-->

Sanfoundry. (2013). C Program to List Files in Directory. [online] Available at: <https://www.sanfoundry.com/c-program-list-files-directory/>

scanftree.com. (n.d.). C Program to count number of characters in the file — Basic , medium ,expert programs example in c,java,c/++. [online] Available at: <https://scanftree.com/programs/c/c-program-to-count-number-of-characters-in-file/>

Singh, R. (2018). How to create an array from user input in C. [online] Interview Sansar. Available at: <https://www.interviewsansar.com/how-to-create-an-array-from-user-input-in-c/>.

TutorialKart. (n.d.). How to Delete File in C? [online] Available at: <https://www.tutorialkart.com/c-programming/c-delete-file/>.

TutorialKart. (n.d.). How to Rename File in C? [online] Available at: <https://www.tutorialkart.com/c-programming/c-rename-file/>.

TutorialsPanel. (2019). Delete a specific line from a text file using C. [online] Available at: <http://www.tutorialspanel.com/delete-a-specific-line-from-a-text-file-using-c/index.htm>.

www.gnu.org. (n.d.). Directory Entries (The GNU C Library). [online] Available at: https://www.gnu.org/software/libc/manual/html_node/Directory-Entries.html.

www.studytonight.com. (n.d.). C program to Display Current Date and Time — C Programs — Studytonight. [online] Available at: <https://www.studytonight.com/c/programs/misc/display-current-date-and-time>.

www.techonthenet.com. (n.d.). C Language: atof function (Convert String to Floating-Point). [online] Available at: https://www.techonthenet.com/c_language/standard_library_functions/stdlib_h/atof.php#:~:text=In%20the%20C%20Programming%20Language.